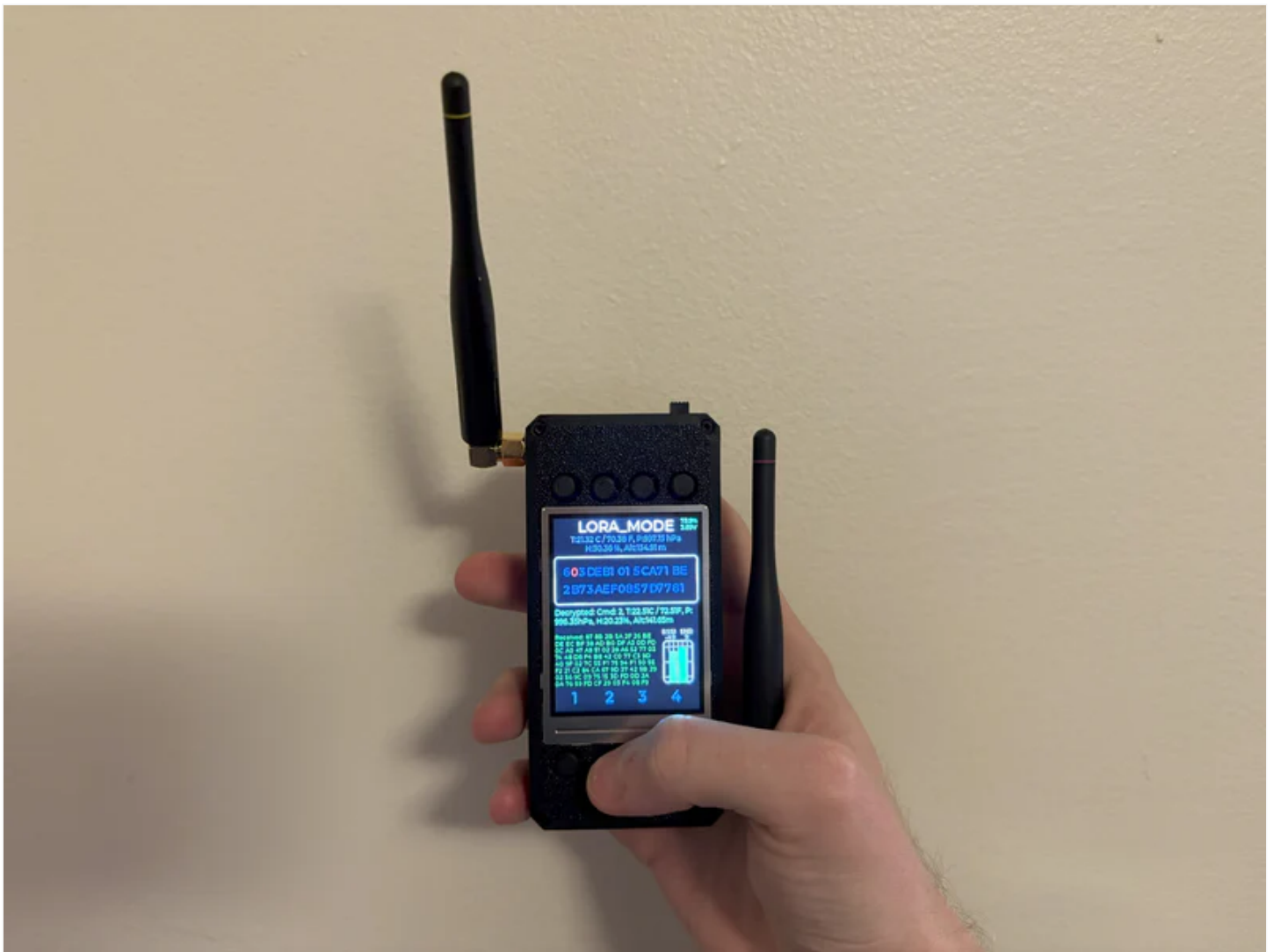


► Table of contents

A Quick Preview

Presenting **DualCast**! My latest (and most advanced) project to-date. It is a compact wireless device capable of analyzing live traffic on 2.4GHz **Wi-Fi networks** in addition to being able to send 915MHz [AES-128 encrypted](#), long-range commands via [LoRa technology](#). (Up to over a kilometer on default settings!)







Flip 180° to activate Wi-Fi mode.



In addition, it comes with numerous sensors such as a BME280 for atmospheric sensing, a BMI323 for orientation detection, and a VL53L4CX **LiDAR** for quick precision distance measurements from 0-6 meters. *(I just thought, "why not!")*



DualCast | Network Analyzer and LoRa Transceiver

\$174.00

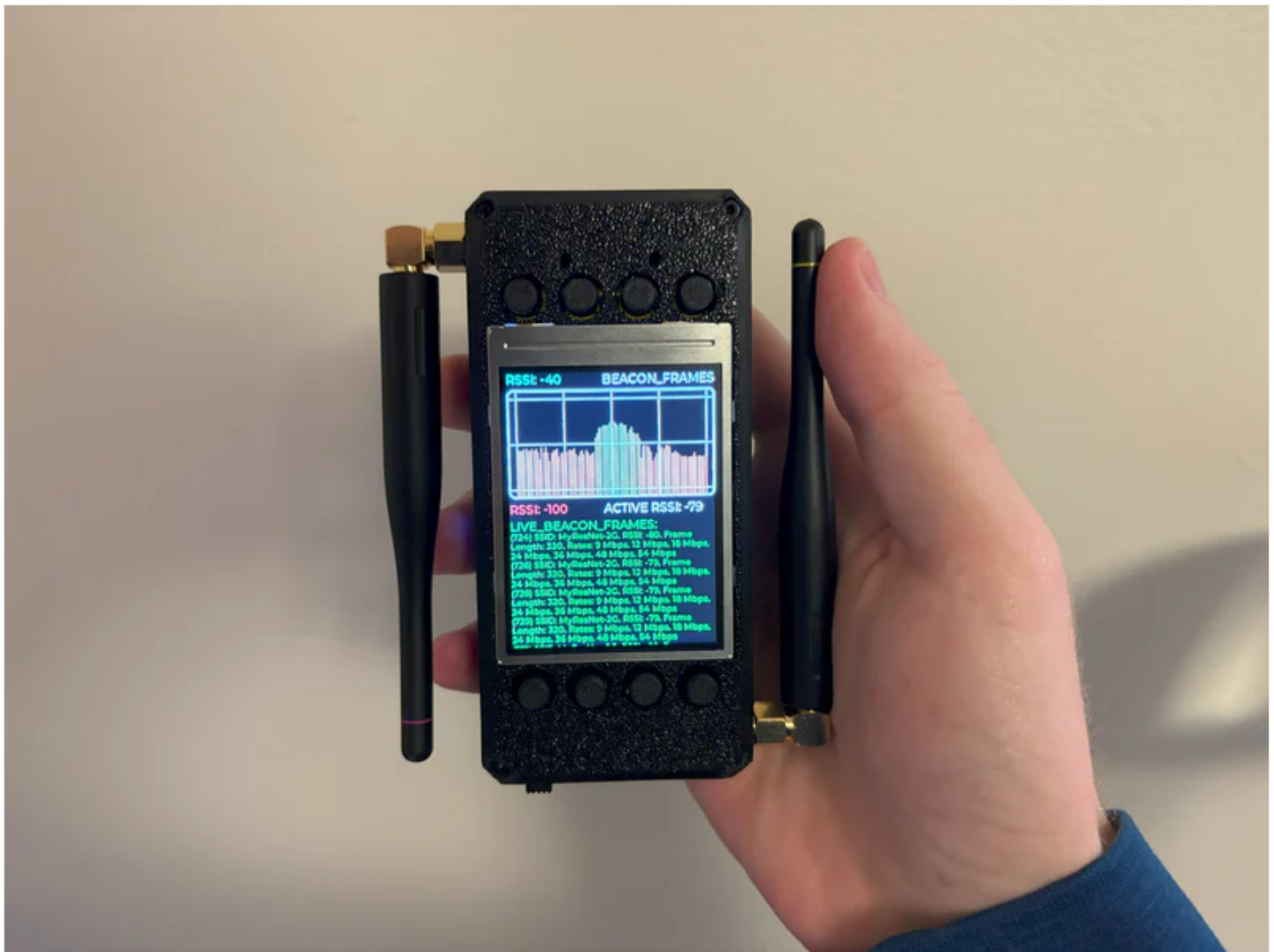
[Go to shop!](#)

I've been super impressed with the LiDAR, as it's a nice upgrade over the one I used in my [smartwatch project](#). I've been able to get basically identical distance measurements when comparing to a tape measure indoors. It also comes with a lot of additional data to make sure you have everything needed for total measurement accuracy, such as ambient light, tolerances, etc. *(All authentic data from the ST VL53L4CX SDK!)*

The device's dual-broadcast capability as a Wi-Fi-LoRa hybrid also allow it to prove useful in various wireless circumstances. In analyzer mode, you can simply scan for active 2.4GHz networks and then select the one you want to analyze. From there, you can listen for live [Beacon](#) or [Data](#) Frames to analyze RSSI, active device MACs, destination MACs, frame lengths, and data rates.



You can scroll up and down too for easy reading!



Beacon Frames



Data Frames

This can be incredibly useful in analyzing who all is communicating with the network (in case you're worried your neighbor has been stealing your Wi-Fi), or in identifying network traffic to determine the fastest network. It also works great for if you just want to find the spot in your house with the best signal strength to set up your desktop. I'll go into the details of how this works later.



One antenna is optimized for 2.4GHz (Wi-Fi), the other is optimized for 915MHz (LoRa).

When you're done, simply flip DualCast 180 degrees (literally) to put it into LoRa mode. In this mode, you can communicate with your own weather or other sensor stations, dispatch commands (lock/unlock door, turn on/off lights, etc.), or any other applications where an independent long-range signal would be useful. You can kind of think of it as an IoT command center, in which you can communicate with all your other custom devices securely. Just simply click the command you want to send. (Up to 255!)



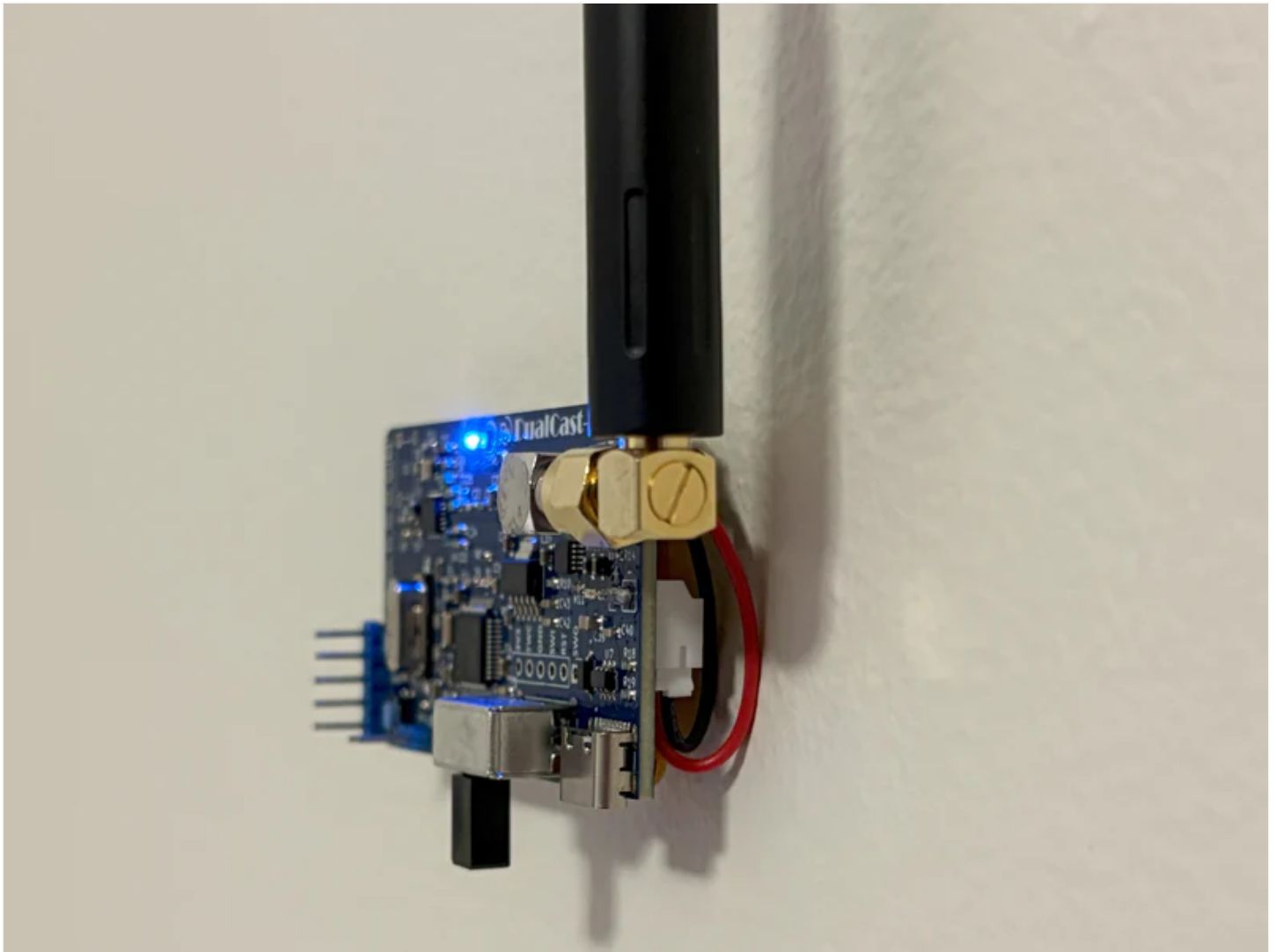
To make this easier for you, I also built a custom LoRa receiver ([open-source](#)) designed to decrypt the AES-128 long-range signals sent to it by DualCast and perform whatever task you want it to do. But the best part is that you can use it without touching a single line of the source code!



By default, this is achieved through basic GPIOs. On DualCast (the transmitter), just simply send a number. This number will then be encrypted based on your unique key typed into it, to ensure that only you can send the command. When the receiver gets the message, it will decrypt it using that same key (which you tell it beforehand) and then set the corresponding GPIOs high (corresponding to the number you sent it in binary). From there, just connect an Arduino, ESP32 DevKit, or whatever else to the GPIOs and when that sequence goes high, have it execute your command. EZPZ! I'll go into more detail on how this works later in the article.

[Example Arduino sketch here.](#)

Although, if you'd prefer to just use the receiver directly that's perfectly fine too! For this reason, I've broken out pins for UART, I2C, and PWM to interface it with whatever you desire. In addition, it can also easily connect to a battery for rural commands should you need them.





Rechargeable!

But what if you want to do a one-to-many broadcast? Not to worry, just have your other receivers use the same key and have each listen for whichever unique command they're waiting on. (Ex: *The one in your kitchen is listening for a 5, while the one in your bedroom is listening for a 28.*)



After a long day of sending commands/collecting sensor data and exploring the world of wireless, the built-in **L**ithium-ion **P**olymer (LiPo) battery on DualCast can be safely and easily recharged via a USB-C cable just like on the receiver.



And of course, like all my other projects, DualCast is [completely open-source](#) should you want to make any edits to tailor it to your own desires. For this reason, I also added some extra GPIOs that could be used for any various purpose you might want them for.

Now let's get into how this project works!

Working Concept

The main brain of this project is a [STM32H563 microcontroller](#) (MCU). Its job is to communicate with all the various sensors/ICs on the **Printed Circuit Board** (PCB) to integrate everything together and make the device work.

To make this easier, the STM32 is running [FreeRTOS](#) which is a [thread-based operating system](#) that makes managing everything far more straightforward and legible.

To handle the wireless analyzer aspects of this project, I also included an [ESP32-C6](#) which communicates with the STM32 over [UART](#). Basically, what happens is the STM32 will first enable the ESP32, then make a request. The ESP32 fulfils the request then sends the results back to the STM32. From there, the STM32 re-disables the ESP32 and then processes and displays the data.

If you're wondering why the ESP32 isn't just running the show in its entirety rather than the STM32, it is because the ESP32 is far more power hungry than the STM32, so it's best not to have it running at all times. There also just aren't enough pins on the ESP32 to be able to handle everything.

For sending and receiving through LoRa, I'm using an [SX1262](#), which is a cool Sub-GHz LoRa transceiver. To communicate with it, I'm using [SPI](#) since it natively supports it. This is what allows DualCast its ability for super far independent transmissions!

For performing the encryption, I'm using [AES-128-CBC](#) which stands for **A**dvanced **E**ncryption **S**tandard (**128** bits) with **C**ypher **B**lock **C**haining.

How this works is each device (transmitter and receiver) is assigned the same private key that is 16 bytes long. (This would usually be generated randomly and then preprogrammed into the receiver, in which you would then type into DualCast.) Right before a message is sent, a public 16 byte IV (**I**nitialization **V**ector) will be randomly generated for each message and then used to further encrypt the message, ensuring uniqueness each time. Then, the encrypted message will be sent along with the unique IV. The receiver then receives this, extracts the unique IV from the message, and then uses that and the private key to decrypt the payload. Pretty cool! ([More info here.](#))

What the function looks like.

Needless to say, to break this encryption through brute force would be basically impossible with 2^{128} different possibilities ($3.402e+38$). That's a big number! (Assuming you don't spill your 16 byte encryption key after it comes up in casual conversation.)

To help keep this secret, DualCast also has it hidden by default on the LCD. This way, whenever you just want to send something and aren't actively messing with the key, you won't have to worry about someone seeing it while walking by or if someone is recording around you.

The cool display I'm using this time is a ST7789VI 2.4" TFT LCD (**L**iquid **C**ystal **D**isplay) communicating over [SPI](#). To manage it, I'm running [LVGL](#), which is an awesome graphics library based in the C programming language.

How DualCast Analyzes Networks

So I mentioned earlier that DualCast can identify active network users (among many other things). **This is true!** Wireless networks work by utilizing a constant stream of communication between your device and the router. This communication, going back and forth in "packets", will always include some information to identify the sending and receiving device (their MAC addresses) in addition to some other info like their data length, etc.

Image from cyber.gov.au

DualCast utilizes this information to collect and tally the active network users by listening for these packets and recording them when they are heard (**their unique MAC addresses**). With this information, you'll be able to identify who and how many people are currently exchanging data with the network. In addition, you'll also get some useful information such as their **RSSI** (signal strength) relative to you and their **frame length** (how much data is being sent as each packet). Although, like I said, the devices are being recorded as their MAC addresses, meaning that's all you'll be able to see about them specifically. (It's not like you'll see "John Smith is active on his iPhone 10.") However, this also has many benefits.

Image from askleo.com

For one, most users who connect to familiar networks ([WPA2+](#)) will always have the same MAC addresses (fixed Wi-Fi address is the default setting for iPhones). This introduces many interesting possible applications, such as determining where someone is (but not in a bad way). Say for example, you own a coffee shop and want to determine how many repeat

customers you have. When customers connect to your coffee shop Wi-Fi network, they will exchange communication with it in which their addresses could be heard. You could then listen for this to get a sense for how many customers are repeat customers who have had that same MAC address heard on a different day. Pretty neat! Although, they will always have a layer of security behind their MAC address, so you'd never know who they are, only that they are there. (Unless you take all their phones and look up their MAC addresses, that is.)

Three active network users detected!

Building on this further, even iPhones/scanner apps are not allowed to display device MAC addresses since iOS 11. **But DualCast can!** So what if your neighbor is stealing your Wi-Fi? Well, if you notice a weird MAC address on your network and get paranoid you can go over to their house and ask to see their phone's "Wi-Fi address" (MAC). Although, it would also be likely that the weird MAC is a smart dryer or something that uses Wi-Fi, so maybe don't go to far haha. A better application might be if you have a kid and want to know if they are using their phone past bedtime. All you have to do it go to "About" on their phone sometime prior to get the device MAC address and then if you see excessive data packets being sent from that MAC address past bedtime you'll know something's up.

So what I've been mentioning up until now are data frames (packets containing data being sent back and forth between device and router). But DualCast can also analyze beacon frames! Beacon frames are packets being sent out by the router only, announcing that there's a network that a device can connect to.

Beacon frames are a bit less exciting, but have a majorly useful application. Because it is the router that is sending these packets out, the RSSI (signal strength) received will be relative to the router. This means that you'll be able to use them to determine where the best signal strength is, great for finding the perfect place to set up your desktop! (Or whatever else.)

You can walk around to get some variety.

Another cool experiment you could do is just generating some network traffic on your phone or PC and then using DualCast to check it out. For example, in doing this I've learned some interesting information about when packets are sent and held and in what quantities. For instance, something like a simple network page may not always trigger a packet being "sniffed"

(detected) but I can normally always see a bunch when loading a YouTube video or streaming music.

Even one time, I was puzzled when not seeing any network traffic when streaming YouTube since my computer claimed it was connected to the network that I had selected to analyze. Although, it had turned out that my computer was lying to me and was actually connected to a different network and was just displaying the wrong SSID. Interesting!

Using DualCast, I was also able to notice that data frames would only get sent on YouTube when buffering new video (the little gray bar) and normally in high quantities. I thought this was quite neat, considering that when using a music service such as Spotify, data frames would be sent almost constantly.

Understanding The Hardware

Here are the full project schematics for your reference.

DualCast:

Licensed under: <https://github.com/RoboticWorx/DualCast/blob/master/LICENSE.md>

LoRa receiver:

Licensed under: <https://github.com/RoboticWorx/DualCast/blob/master/LICENSE.md>

Let's break it down. I'll just go over the main DualCast schematic, since the receiver is basically just a copy with some minor changes.

To set up the STM32H563 or any MCU, you should always first look through [the datasheet](#). From there, you can navigate to the power supply section to figure out which decoupling capacitors are needed where, and then to the crystal specifications page. In my case, an 8MHz crystal worked well as it was within the suggested range. In addition to this, I also added some pull-up resistors on the I2C lines (required) and the needed pull-up/pull-down resistors on the BOOT and RST pins to determine the default start-up state. The 0-ohm resistors on the USB

pins are so that they can be placed only when uploading to the STM32, since USB is also shared with the ESP32-C6.

Speaking of ESP32-C6, let's look at that next!

Like before, the first step is to check out [the datasheet](#). However, Espressif (the company behind ESP32) also has [this fantastic development page](#) that goes through everything necessary for the schematic in detail. There is also a [PCB design guidelines page](#) for when designing the layout. This said, I won't really explain much else here, since everything needed is included in those links. You should check it out!

The SX1262 schematic was probably the toughest because of this complex matching network. On [the datasheet](#), a matching network is mentioned but not really sized or specific since it depends on the frequency you're broadcasting at (changes based on region). To get the correct values for where you are (in my case, 915MHz in the US) I looked up the [SX1262 documents](#) page.

Home > Products > Wireless RF > ICs for Sensors > SX1262

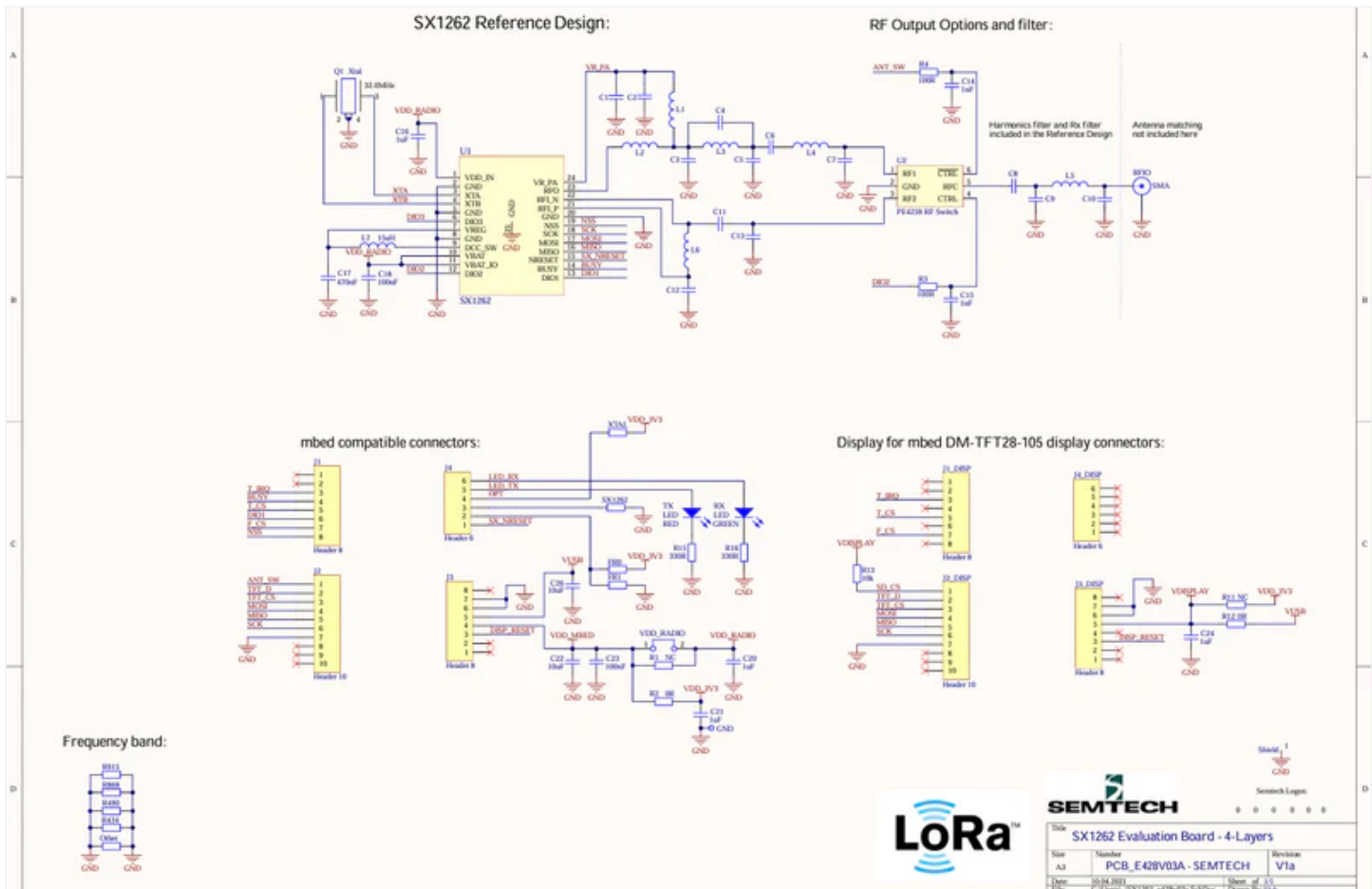
SX1262
LoRa Connect™ Long Range Low Power LoRa® Transceiver +22dBm, global frequency coverage

[BUY NOW](#)
[DATASHEET](#)

Overview Documents Dev Kits & Shields Software Resources Support

DOCUMENTS	RELEASE DATE	TYPE	
DATASHEETS			
SX1261/SX1262 Datasheet	2022-01-06	PDF	
APPLICATION NOTES			
AN1200.28: MCU Requirements for LoRaWAN	2019-07-02	PDF	
AN1200.33: Crystal Oscillator Trimming	2023-10-10	PDF	
AN1200.35: Software Manchester Encoding for the SX1261/2	2019-06-05	PDF	
AN1200.36: RxDutyCycle with the SX1261/2	2019-06-05	PDF	
AN1200.37: Recommendations for Best Performance (thermal relief)	2019-07-02	PDF	
AN1200.40: Reference Design Explanation for the SX1261/62	2019-07-02	PDF	
AN1200.48: LoRa Channel Activity Detection (CAD) with SX126x	2023-10-10	PDF	

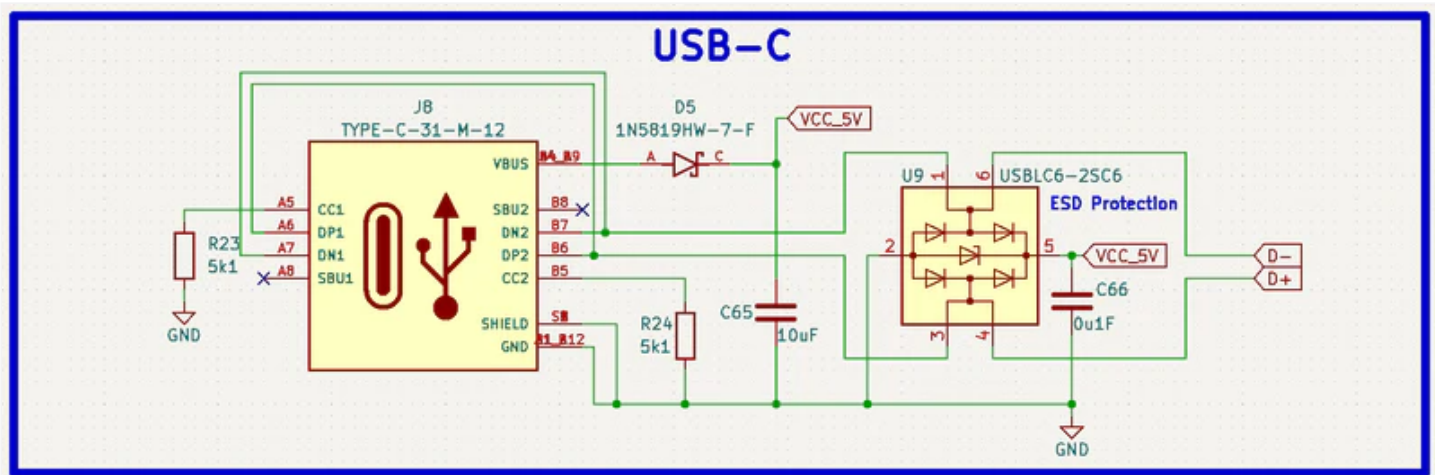
From there, there's a TON of information. To avoid having to look through all of it, I was most interested in the [reference design explanation](#), [PCB design guidelines](#), and the example [915MHz reference design](#). Based on the example (since it matches my region), I copied everything over and sized the components using the included BOM.



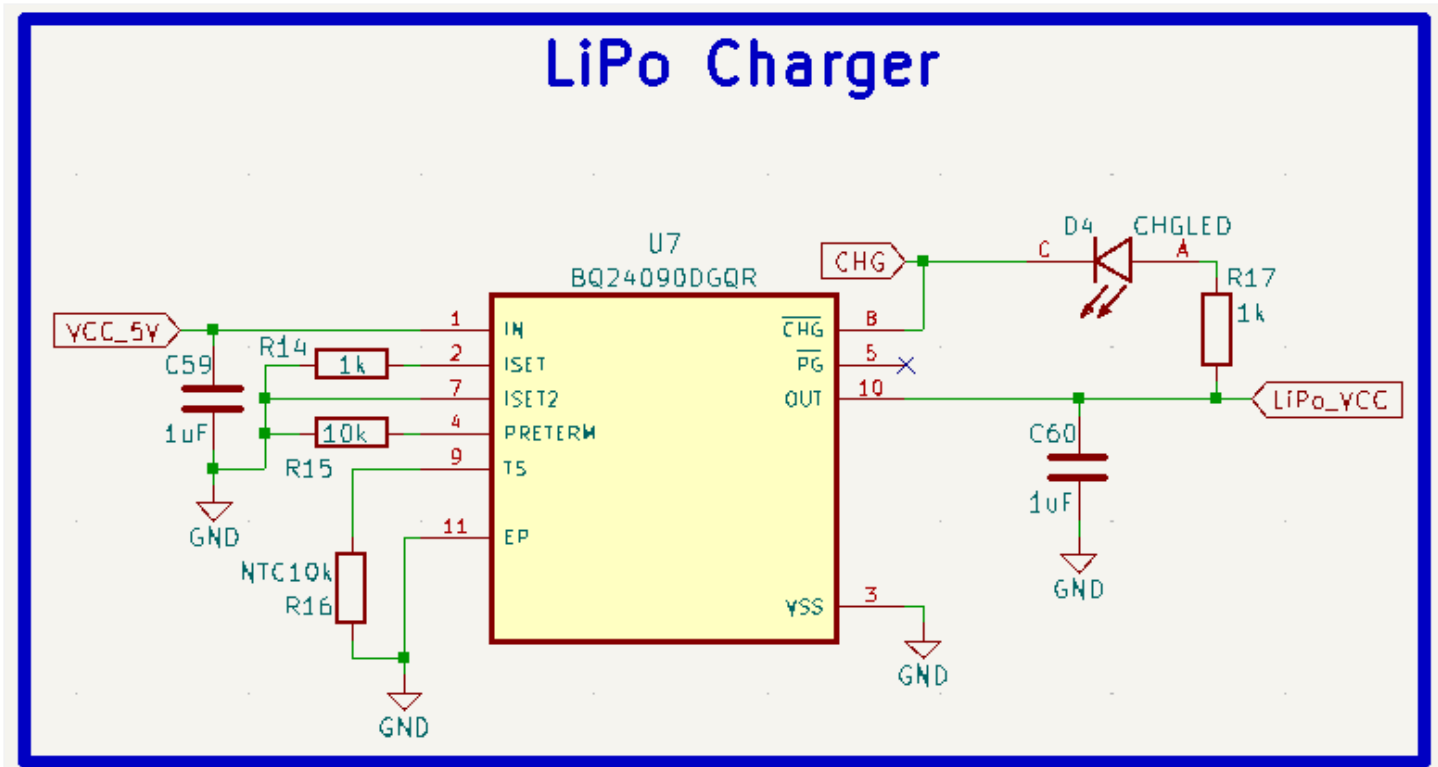
Semtech Reference Design

Great! After verifying that I moved everything correctly, I just added the antenna pi-matching network for a 50-ohm antenna (since the reference design can't anticipate what antenna I use), an extra ESD protection diode, and connected up the pins. [More on pi-matching here.](#)

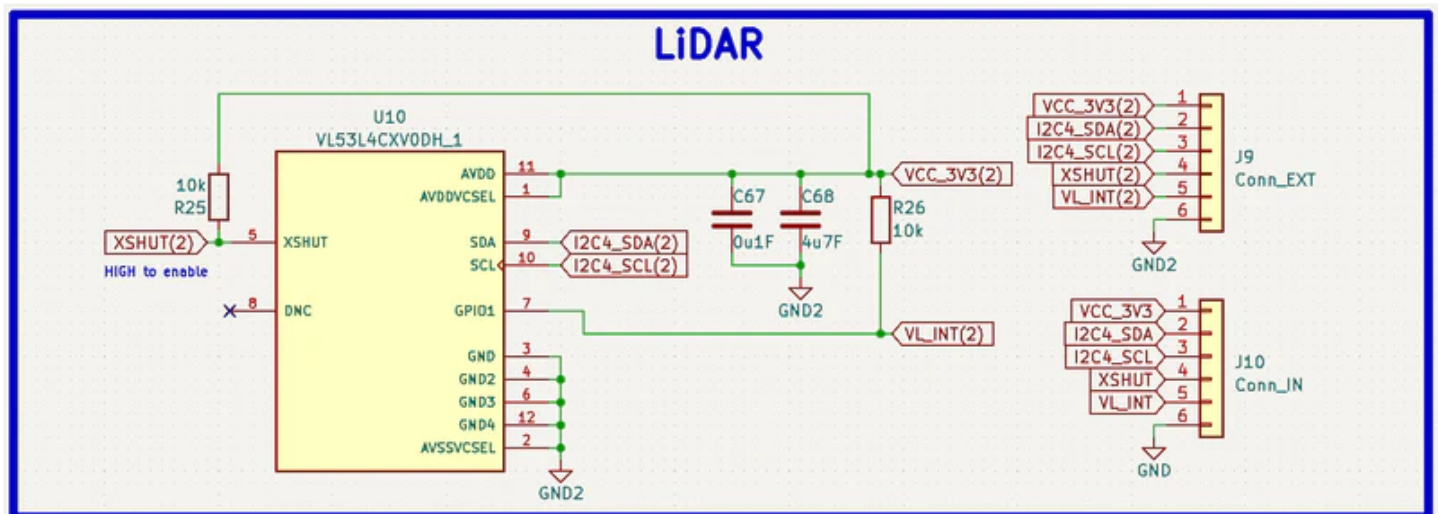
Now let's move onto the rest.



A small change I made to the USB-C connector circuit is that I'm now using this cool ESD protection IC rather than the individual diodes (which were tiny). This just makes it a bit easier to assemble.



For the LiPo charging circuit, I just used my typical setup with the [BQ24](#). Works great! R14 and R15 set the charge termination and speed, with a thermistor for R16 in case the battery gets too hot. Then just a couple decoupling capacitors to ensure a steady voltage.

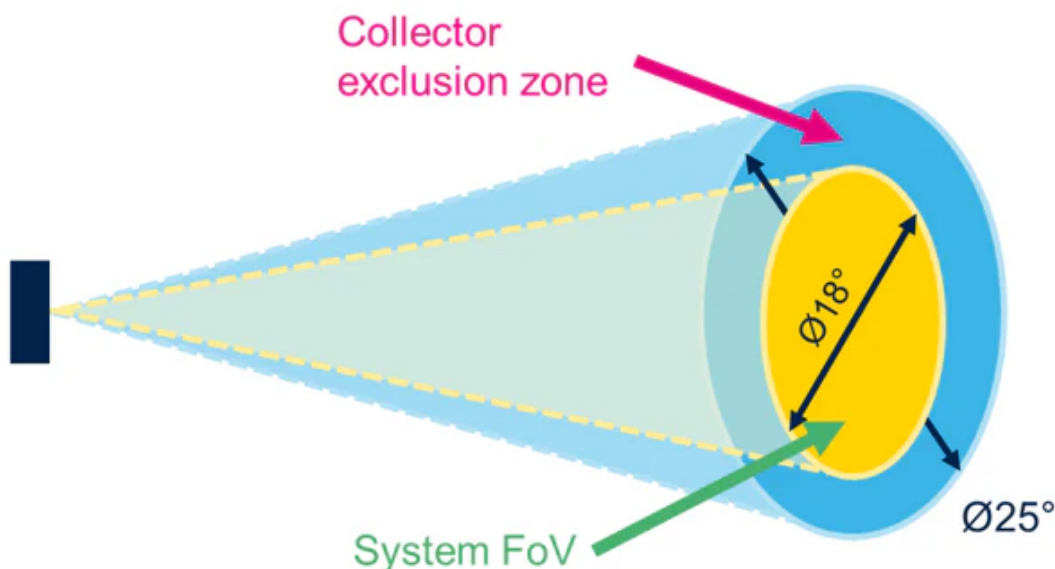


For the LiDAR, I similarly have some decoupling capacitors (sizes recommended by [the datasheet](#)) and then two pull-up resistors, one to set the device enabled by default and the

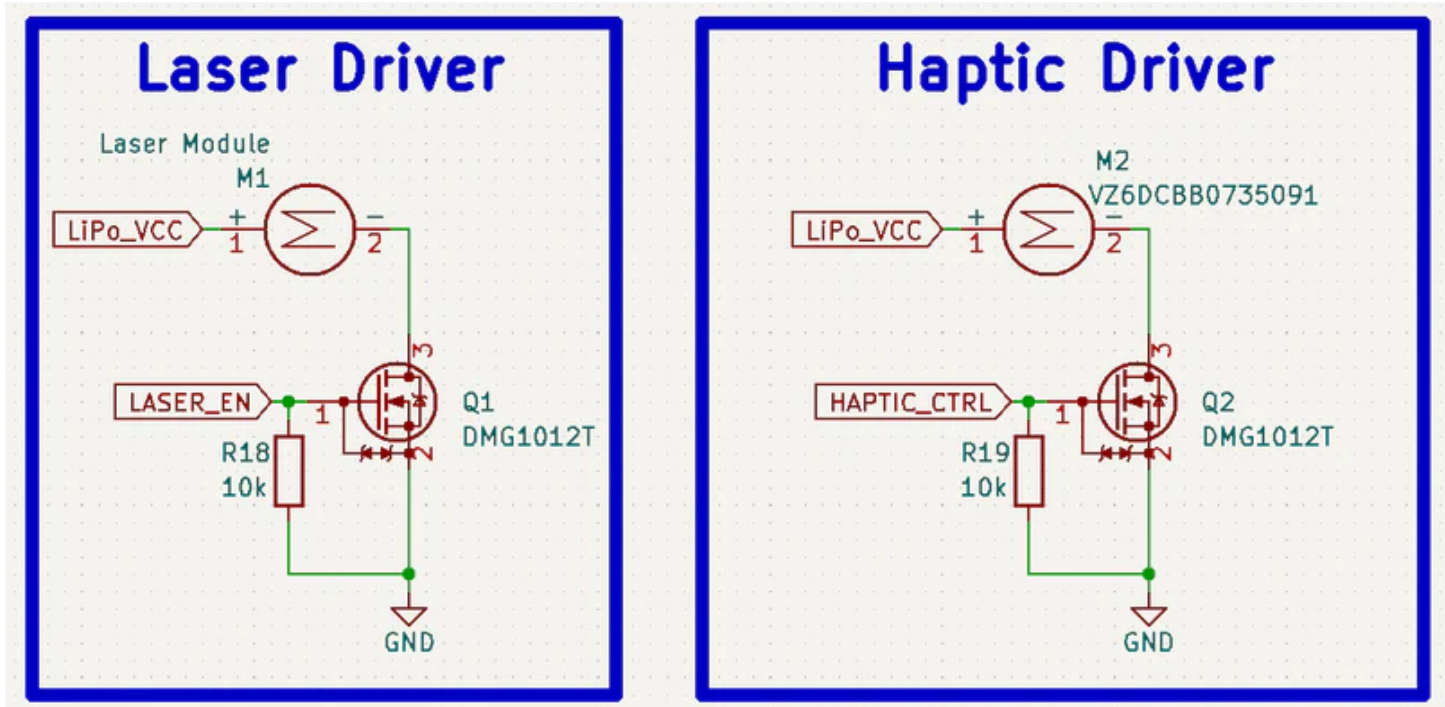
other as recommended. The two connectors on the right are just because it needs to be mounted vertical to the PCB and therefore needs to be connected separately.

I should also note here that the LiDAR measurements are not taken in a straight line like probably implied by the laser! This LiDAR utilizes a surface field of view (FOV) to detect the infrared light being sent back to it in order to take the distance measurement. (The laser is just an aim-assist.) This means the LiDAR works best for measuring distances to objects in which the full FOV can fit on the object. I should also mention that ambient light plays a role in the distance measurement accuracy, so I probably would not recommend it for use outside on a sunny day, as the sensor will have a hard time differentiating the emitted infrared light from that of the sun.

Figure 22. System FoV and exclusion zone description (not to scale)



The BMI323, BME280, and TPS voltage regulator don't require much explanation, I basically just connected up the pins and added the recommended decoupling capacitors to ensure a steady power supply. I should also mention that **the internal BME280 will heat up some due to the surrounding components after being powered on for a while**, so I would recommend only trusting the temperature measurement from when first turning it on. (Before it has time to heat up, which would be accurate.)



For turning on/off the aim-assist laser and haptic motor, I'm just using two MOSFETs with a pull-down resistor on each to make sure they are fully off when not actively being driven. This just acts as a more powerful on/off, as when the MCU pin goes high it will allow current from the load to flow to ground.

That's basically it!

Optimizing LoRa

Here I'll explain further about how the LoRa capabilities on DualCast work, in addition to how you can maximize its range.

As I mentioned earlier, LoRa transmissions are being handled by a SX1262 with a default payload size of 80 bytes at 915MHz. (64B for ciphertext and 16B for the IV. 915MHz is the license-free sub-gigahertz radio frequency band for the US.)

The basic LoRa parameters are as follows:

```
sx126x_mod_params_lora_t lora_mod_params =  
{  
    .sf = SX126X_LORA_SF9,  
    .bw = SX126X_LORA_BW_125,  
    .cr = SX126X_LORA_CR_4_5,  
    .ldro = 0,  
};
```

- **SF9:** The spreading factor determines how many symbols represent one data symbol. A lower SF (SF7) lets you send more data but at a shorter range, while a higher SF (like SF12) increases range and sensitivity at the cost of lower data rates and longer time on air. (Takes longer to send and receive stuff.)
- **BW125:** This sets the channel bandwidth (125 kHz). A wider bandwidth allows for higher data rates but typically reduces sensitivity.
- **CR4/5:** The coding rate is used for Forward Error Correction (FEC). It indicates that for every 4 bits of useful data, an extra bit is added for error correction (4/5). A higher FEC (more error bits) improves reliability in noisy conditions but reduces the data throughput.
- **LDRO:** Low Data Rate Optimization is used when the symbol duration is long by compensating for clock drifts. In this case, it's disabled because the current settings (SF9) with 125 kHz bandwidth do not require it. (Fast enough where it's not necessary.) This should probably be turned on for anything higher than SF10 though.

As the default, I went with SF9 and the other settings above since they were a good compromise between range and the time on air. DualCast wouldn't be as cool if there was a multiple-second delay between sending and receiving stuff.

Graph from a ResearchGate study by Absar Ahmar.

Separate calculation for 80 bytes.

452ms ended up being a bit short (message got cut at the end), so I ended up using 475ms instead, which I felt was reasonable for the waiting time while also keeping some solid range.

(475ms for the command to be received, another 475ms to get the receipt back. So it's 950ms if you're talking round-trip, like requesting data from a sensor.)

*Graph from a Semantic Scholar study by Albert Potsch. **NOTE: These range values will change a lot depending on where you're sending from! (If you're in a rural or urban area, etc.)***

These figures check out with DualCast since I got similar values while experimenting with it in a city. Although, I think you could probably get up to ~5km if you were to use it in a plain line of sight from the receiver in a quiet area (no interfering signals).

This said, however, maybe you're hoping to send data further to a sensor hub you built, or 475ms is too long for the rapid communication you require. Not to worry, just update these settings however you see fit to maximize the range and data rates for your application! The open-source code [can be found here](#). **Just know you'll need to update the receiver to match!**

The SX1262 datasheet can also be [found here](#) if you want to explore the settings further.

DualCast 5GHz

If you like this project but are bummed that it isn't capable of analyzing 5GHz Wi-Fi networks (2.4GHz only), I have some good news. This project uses the ESP32-C6, as it's one of the newest Wi-Fi SoCs released by Espressif, although they do not currently have any chips with 5GHz capabilities.

However, this is set to change in the future with the ESP32-C5 being set to release to markets sometime later this year, **which does include 5GHz capabilities**.

If you would like me to make a second version of this project that will be able to analyze 5GHz networks with the ESP32-C5, in addition to some other cool features like integrated Bluetooth and Wi-Fi based command sending, please consider showing support for this project, as it'll incentivize me to keep pursuing it. You can do this by supporting my work through [becoming a Plus member](#) (where you'll also get all these [additional perks](#)) or by purchasing a DualCast unit from [my shop](#). This certainly goes a long way, as I don't use ads and prototyping these projects can be quite expensive and time-consuming. I'd really appreciate your support!



DualCast | Network Analyzer and LoRa Transceiver

\$174.00

Go to shop!

Some of the different revisions it took to get this project working:

RFoRa was the original project name, but then I thought of DualCast which was cooler.

Building the Boards

On to the PCBs! (**P**rinted **C**ircuit **B**oards.)

The tiny PCB on the bottom left gets broken off and resoldered at the top for the LiDAR.

For this, I did a four-layer design in [KiCad](#) (free) with this stackup:

1. Signal
2. Ground
3. Power
4. Signal

I also included extra ground pours on the top and bottom signal layers where they would fit to help isolate the various signals and add coupling.

- The Gerber/fabrication file for the DualCast board [can be found here](#).
- The Gerber/fabrication file for the receiver board [can be found here](#). (Click "download raw file.")

- The editable STEP and KiCad PCB files are [available here](#) to [plus subscribers](#). Want to [become a plus subscriber](#)? It's only \$5 and I would really appreciate your support!
- The part list (BOM) for DualCast with designators [can be found here](#) with the CPL [available here](#) if needed.
- The BOM for the receiver [can be found here](#).
 - As always, I know that it can seem like a pain to have to order parts, but it's really not so bad as almost all the parts you order from one project get reused. Think about how much you'll learn with some hands-on experience!

If you're interested in learning more about embedded systems (PCB design), check out [Robert Feranec](#) and [Phil's Lab](#) on YouTube. They're great designers and I've learned a lot from them.

To actually build DualCast and the receiver, we will need a PCB manufacturer to produce the boards. For this, I used [PCBWay](#)!

They always have fantastic quality and also offer many other services such as CNC machining, 3D-printing, and PCB assembly. They also feature nine different solder mask colors to choose from and three different silkscreens, which is great for customization. (I even believe they have multicolor solder masks now!) I chose mine to be blue, as I like the color and it offers a slim solder mask clearance, but you can feel free to choose yours to be whichever color you'd like!

Since I designed with KiCad, I didn't even have to leave my design software to check out thanks to their [convenient plug-in](#)! If not though, you can always just go to PCBWay.com, click on quick-order PCB, and upload the [Gerber file](#) for the board. Or alternatively, [just go here](#) which I have saved in my favorites bar. **I recommend clicking the stencil option at checkout to make hand assembly easier!**

Everything else you can leave as the default, unless you have any specific adjustments you want to make. (Ex: lead-free surface finish, remove product number, etc.)

NOTE: You will need to do this for the main [DualCast PCB](#) and the [receiver](#)!

When the PCBs arrived around a week and a half later, they looked awesome!

Being someone who had to go through many revisions to finally get this design right, I always get extra hyped to see them arrive.

To assemble it, first tape down the PCB and then apply [solder paste](#) using the stencil.

With a good paste layer down (it's okay if it takes a few attempts), you can start placing the components. [The BOM includes part designators](#) and can be used as the reference sheet.

NOTE: Make sure the orientation of all the parts (especially ICs) are correct, as pin1 is not always in the top left for this design. Look for the star on the PCB as the pin1 marker. If you can't tell, you can use [this sheet](#). ([This one](#) for the receiver.) Also, make sure the STM32 is aligned correctly, as **it's the concave hole that marks pin1 and not the text orientation!**

Note the STM32 orientation! (Same for receiver).

With that done, you can gently place the PCB on your [hotplate](#) to melt the paste. I normally let mine heat up to 225C before taking it off (depends on the melting temperature of your solder paste).

Epic! You'll have to do this for the receiver too if you want to use the LoRa functionalities.

After that, pop in all the through-hole (THT) components such as the antenna connectors, buttons, power switch, and haptic motor, and solder them on.

After that, carefully break off the LiDAR and solder it on at the top. Do your best to make it perpendicular to the board for the most ideal performance.

You can also take off the yellow lens protector on the LiDAR now.

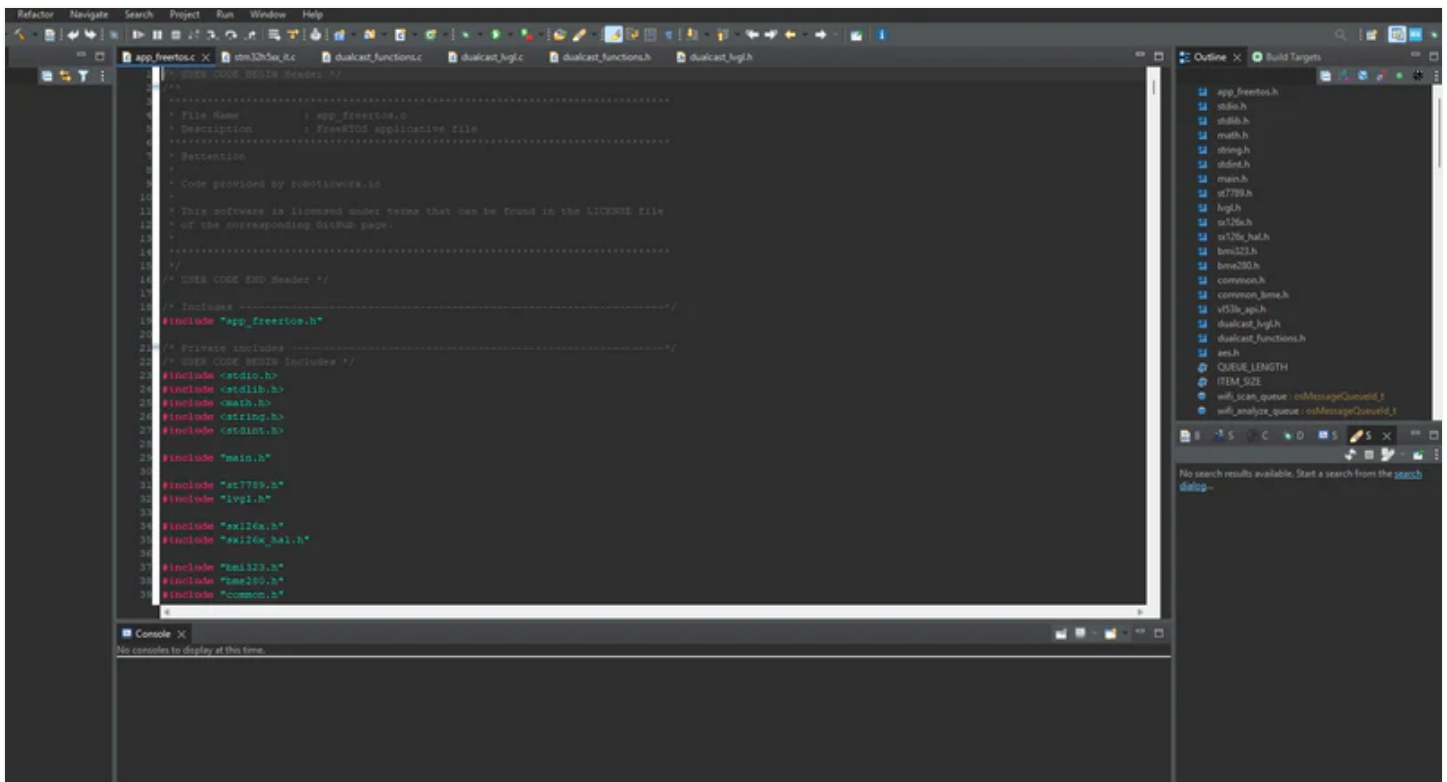
Awesome!

That's all for PCB assembly! Although, before assembling the rest, **I would recommend first uploading the code** (programming instructions below) as a test to make sure everything is working correctly. If everything is as expected (the LCD lights up, etc.) then you can proceed to the assembly section.

Programming

In this section, I'll explain how to program DualCast and the receiver so you can get started using them in your projects. As I mentioned before, I programmed everything here in C using CubeIDE for the STM32 and ESP-IDF for the ESP32. This was mainly because there was a lot of code needed and I wanted to be able to utilize FreeRTOS as well as LVGL and some other C-based drivers.

This said however, I also know that many of you may not be familiar with these development frameworks, so I thought I would explain an easier way to program the boards without having to get into the "include" mess that would be flashing from the IDEs themselves.



If you'd like to go this route, you'll just be flashing the pre-compiled elf/bin files, which will be an exact copy of the final code (which is good). However, I should note that this does mean that **your receiver encryption key will not be unique**, as it will be the default one. This doesn't matter for DualCast since the UI allows you to enter it yourself, but you'll be entering it to match the default receiver key. This should be fine for most applications and will work fine, but just know that it does mean that if someone really wants to they could break the encryption by using the default key and if someone else close to you is also using the default key **there will probably be some crosstalk**. (Note: Any DualCast receivers purchased from [my store](#) will automatically be programmed with a unique key for you to use that will be sent along with the receiver.)



DualCast Receiver | LoRa Decryption

\$64.00

Go to shop!

If you want to avoid this, you can also just [install CubeIDE](#) and copy over [the receiver source code](#). Then, change the encryption key to something unique (located in `main.c`). **Again, this is ONLY for the RECEIVER!** The pre-compiled code is fine for DualCast itself. *If you have include errors, just right-click on the file not being included and then click "add/remove include path" and then with both "Debug" and "Release" selected click "OK". To upload the code, click the green "play"-looking button. (I'll also release a how-to STM32 full tutorial with CubeIDE in the near future.) There is also a lot of info on using CubeIDE online if you have pressing questions. **NOTE: You will also need an ST-Link for uploading, I'll explain connecting it later in this section.***

```
/* USER CODE BEGIN PV */

uint8_t encryption_decryption_key[16] = // CHANGE TO YOUR OWN UNIQUE VALUES
{
    0x60, 0x3d, 0xeb, 0x10,
    0x15, 0xca, 0x71, 0xbe,
    0x2b, 0x73, 0xae, 0xf0,
    0x85, 0x7d, 0x77, 0x81
};

uint8_t command_received;

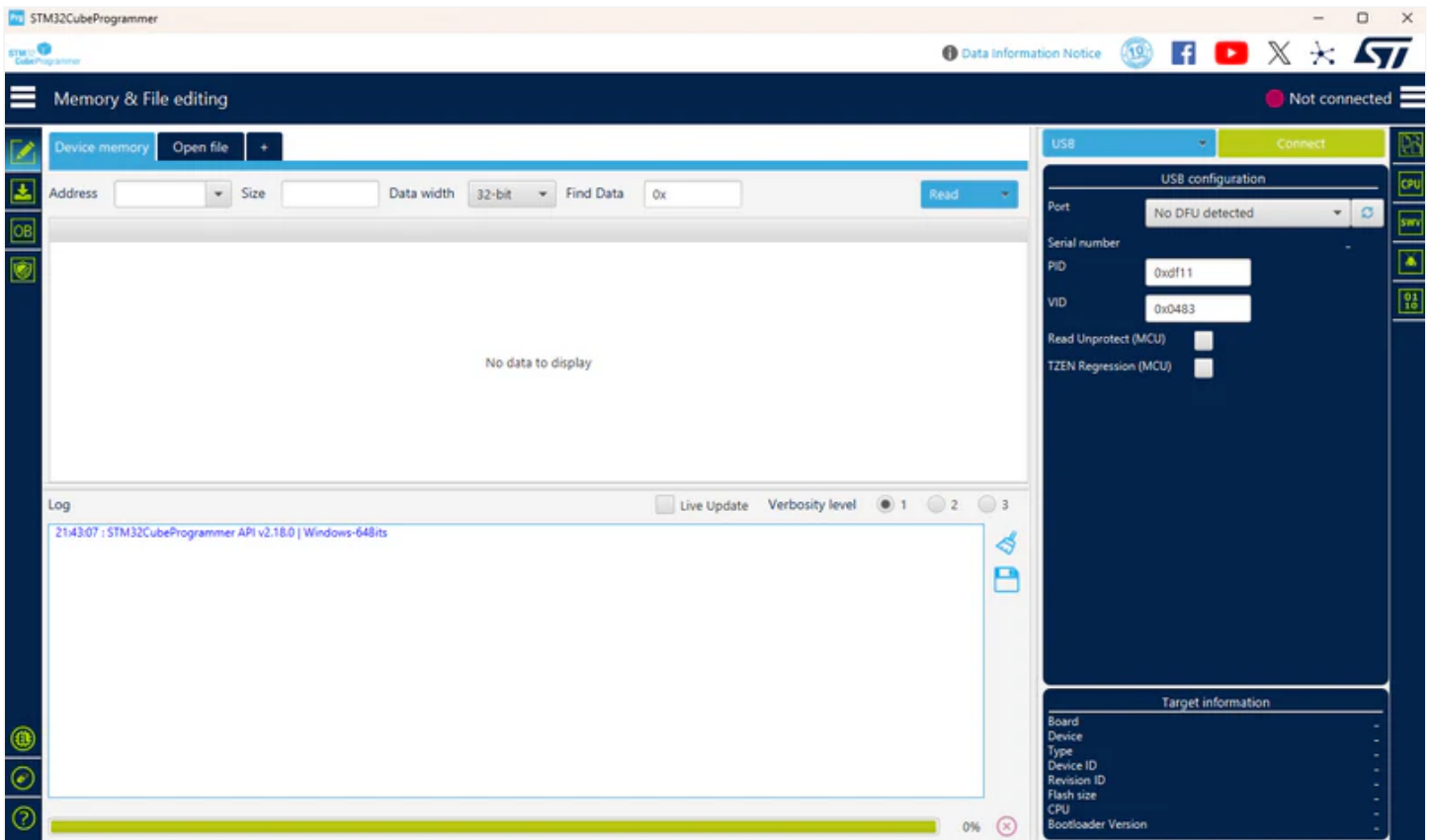
extern bool message_received;
/* USER CODE END PV */
```

*Change this encryption key to something random. **This is what you'll type into DualCast.***

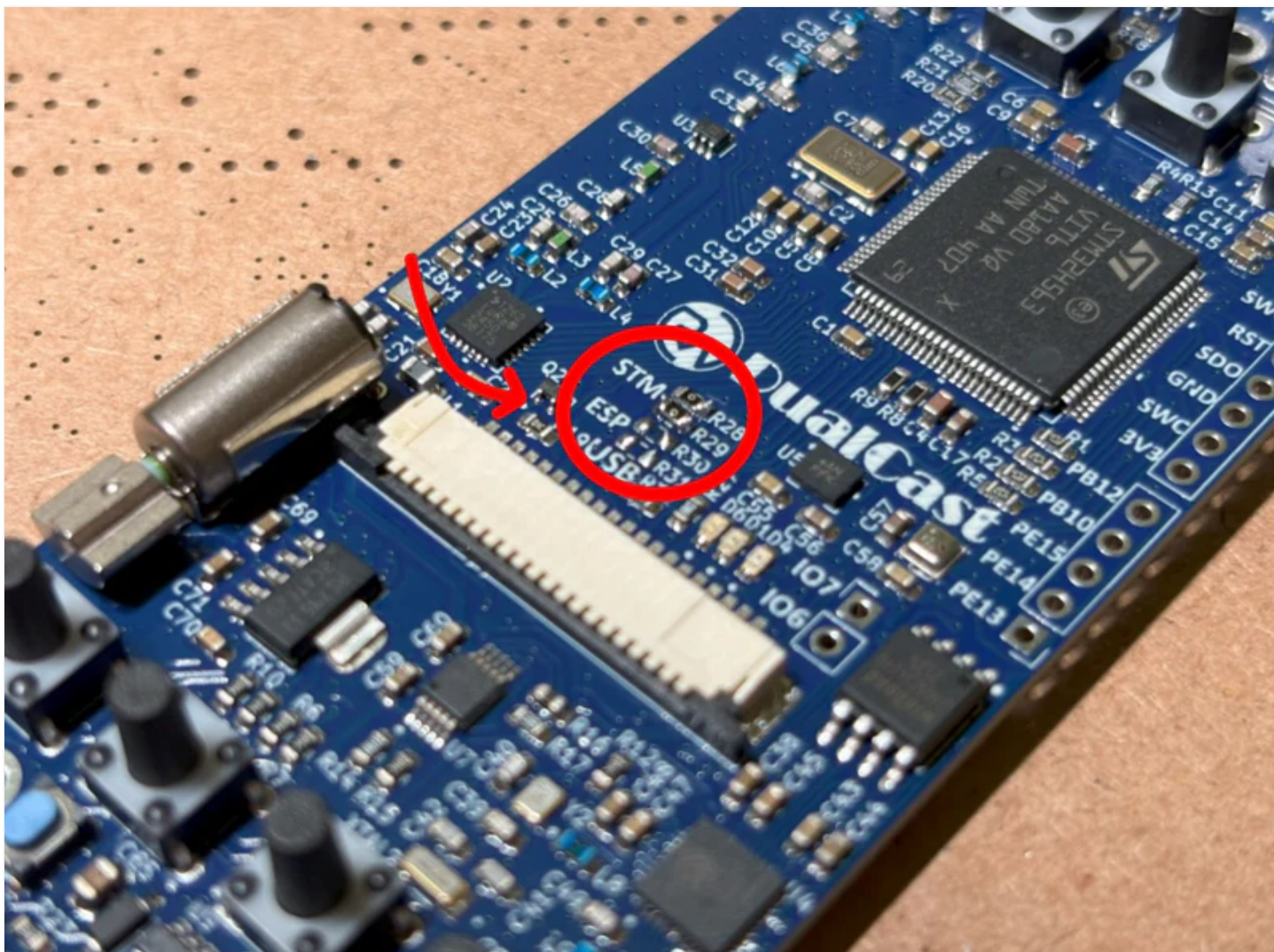
Okay, now I'll explain how to upload the pre-compiled code for DualCast itself, as well as the receiver for if you would like. (Although note what I just said above!)

IMPORTANT: Make sure you do not send any wireless commands (Wi-Fi or LoRa) without the designated antenna attached! If you tell either IC to send something without the antenna connected (you start clicking buttons after upload), the carefully calculated impedances will be messed up, and you could damage the radio with reflected RF leading to less range and poor wireless performance. For this reason, I recommend connecting the two antennas now just in case you press something by mistake after upload. (Yellow-band 915MHz one for LoRa in the top-left; purple-band 2.4GHz one for Wi-Fi in the bottom right.)

For uploading to DualCast, you'll need CubeProgrammer and the ESP32 flash download tool. *You need both of these because DualCast uses an STM32 and an ESP32, which need to be uploaded to separately.*

*CubeProgrammer*

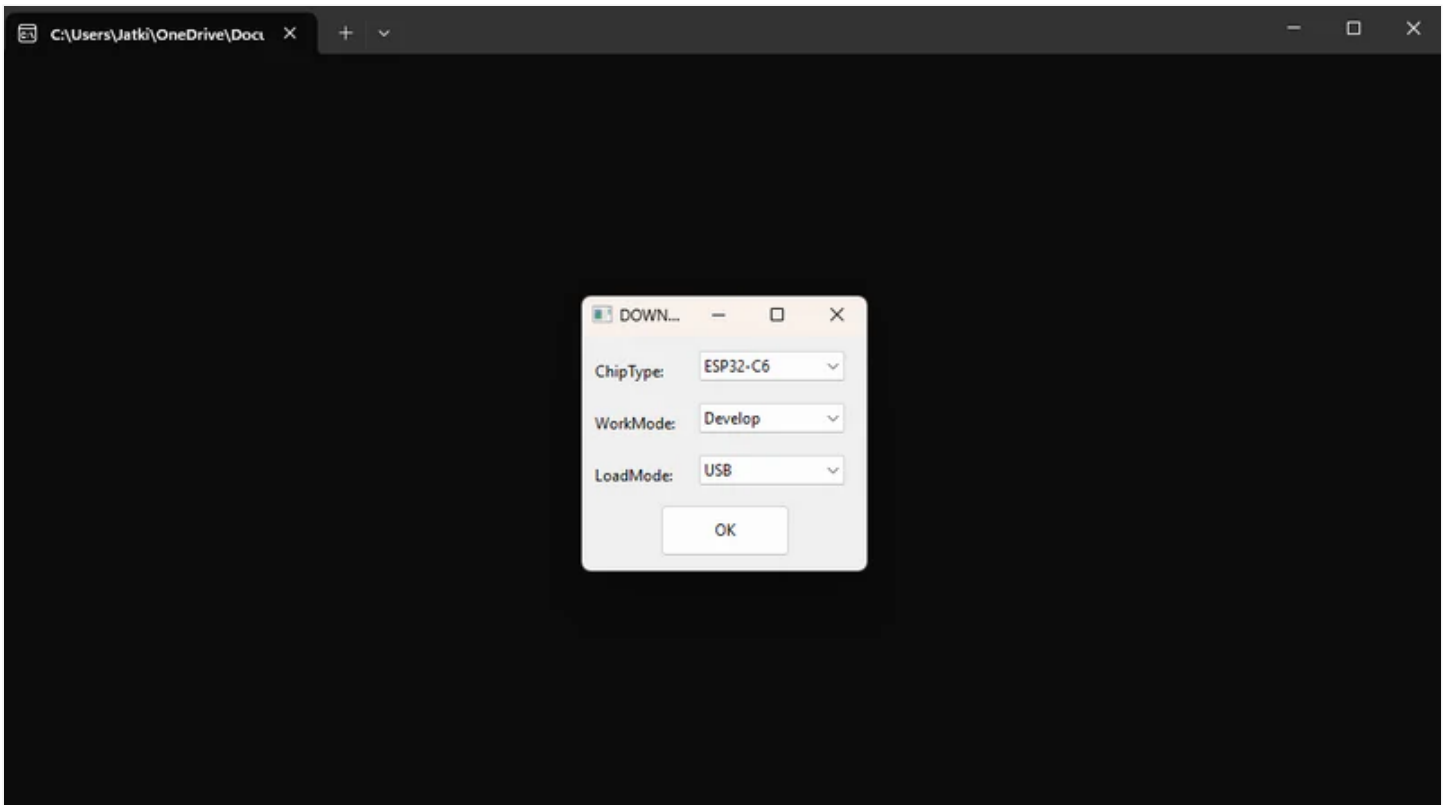
All the needed information about installing and using CubeProgrammer [can be found here](#) from my OpenTemp project. The only difference is that **you need to flash the [DualCast.elf file](#)** (click "download raw") instead of OpenTemp.elf. In addition, **you also need to make sure the USB jumpers are connected for STM32 on the PCB!** This can be done with a 0-ohm resistor or just bridging it with some solder.



Like this!

For flashing to the ESP32, all you need to do is [go here](#) and download Espressif's latest Flash downloader tool. Then, click on the executable (.exe) file and enter the chip information. (I also have an [ESP-IDF tutorial here](#) if you're interested.)

Name	Status	Date modified	Type	Size
bin		6/7/2024 5:29 PM	File folder	
configure		6/16/2024 7:48 PM	File folder	
doc		6/16/2024 7:48 PM	File folder	
logs		6/7/2024 5:28 PM	File folder	
secure		3/19/2024 2:45 PM	File folder	
flash_download_tool_3.9.7.exe		6/16/2024 7:48 PM	Application	20,208 KB



After that, download these four prebuilt files: (Click "download raw file").

- [bootloader.bin](#)
- [partition-table.bin](#)
- [app-template.bin](#)

Then just copy the addresses from the flash_args file into the flash tool. (*Yours will be the same as mine*).

Make sure they are selected and green like above!

Great! Then just put the ESP32 into bootloader mode (hold BOOT > press RST > release BOOT), select the COM port, and hit start.

As an important note: now you need to make sure that the jumpers for ESP32 are selected on the PCB by placing a 0-ohm resistor or bridging the connection with solder. The USB lines for the ESP32 need to be connected **without** the ones for the STM32 connected. In addition, **the ESP32 is only enabled when DualCast is in Wi-Fi mode!** This means you need to have the PCB held upside down when powering on and then connect via USB.

Like this!

Now that DualCast is successfully set up, let's move back to uploading to the receiver.

As I said before, to upload to it, you will need a [ST-Link](#). This is basically just a debugger/uploader which is supported by all STM32s. It is needed here because the NBOOT_SEL bit is checked by default on STM32U0, meaning you cannot do the first upload with USB.

Unchecked enables setting the BOOT mode with pin state (button).

Not to worry, ST-Links are pretty cheap and accessible, and you would have to get one anyway if you ever wanted to do any debugging on future projects. I personally use a [ST-Linkv3](#), but there are also [more affordable versions](#) that would work just fine.

It should be noted these are the corresponding pins on the receiver that you would connect to the ST-Link:

- SWC - Serial Wire Clock (SWCLK)
- SDO - Serial Wire Data Input/Output (SWDIO)
- SWO - Serial Wire Output (this is normally used for print statements and live values)
- RST - Reset
- 3V3 - 3.3V reference voltage
- GND - Ground

When everything is connected, either upload the code with a custom encryption key (as discussed at the beginning of this section) using [CubeIDE](#) and [the source code](#), or with the [pre-compiled .elf file](#) with the default key. If you chose to use the pre-compiled file, follow [the same steps](#) you used for uploading to DualCast, except pick ST-Link instead of USB and use the [DualCast-RX.elf file](#). You also don't need to hold down any buttons when using ST-Link so you can just go ahead and upload the program as soon as it's connected. If you would like additional protection should anyone try to read/reverse-engineer the key off of the flash memory of the receiver, you can also enable RDP, which will lock down the memory. [More about it here](#).

That's it!

Like I mentioned before, also feel free to rummage through the [source code](#) at your leisure, should you want to make any edits, verify security, etc. to personalize this project to your full desires. You could even add an unlock pin for added security should you ever lose it.

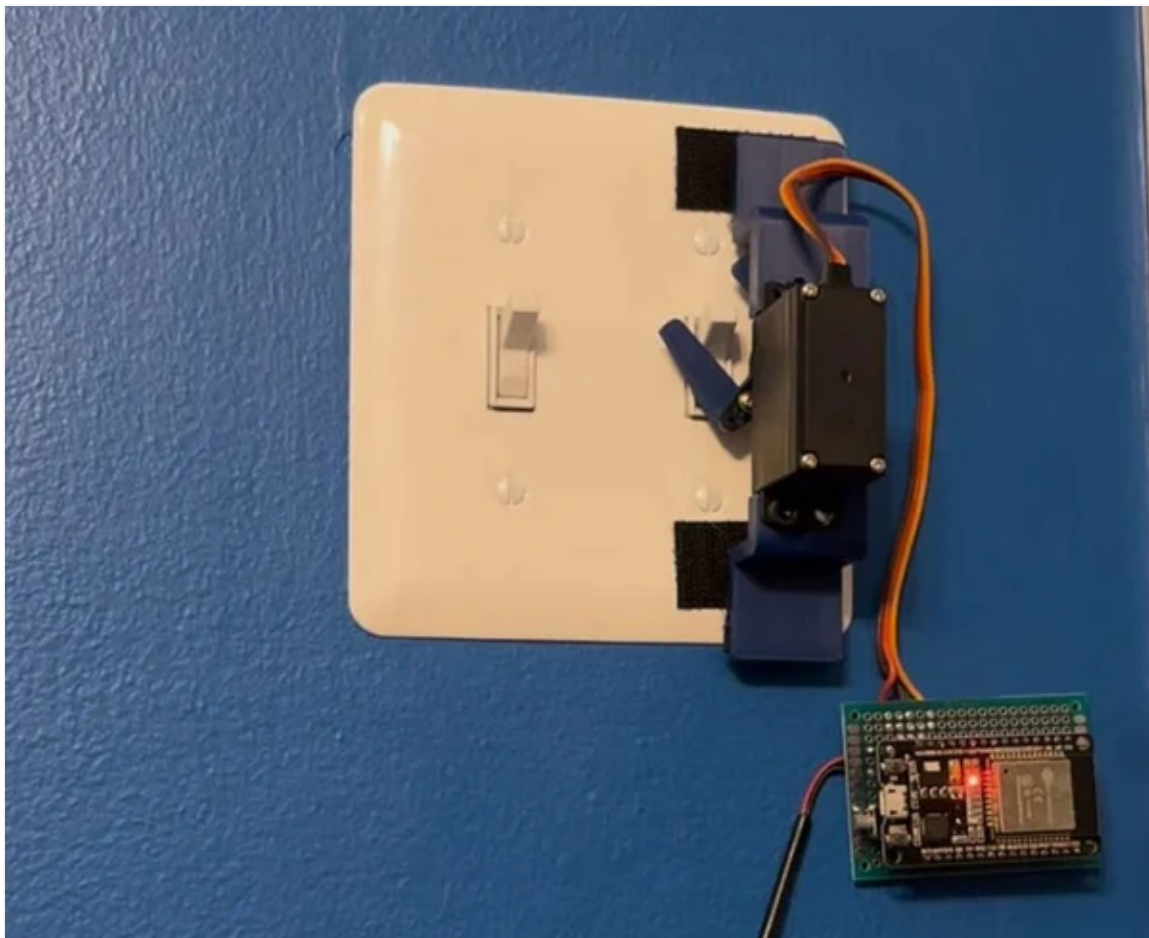
Assembly

This section will go over how to assemble the DualCast casing.

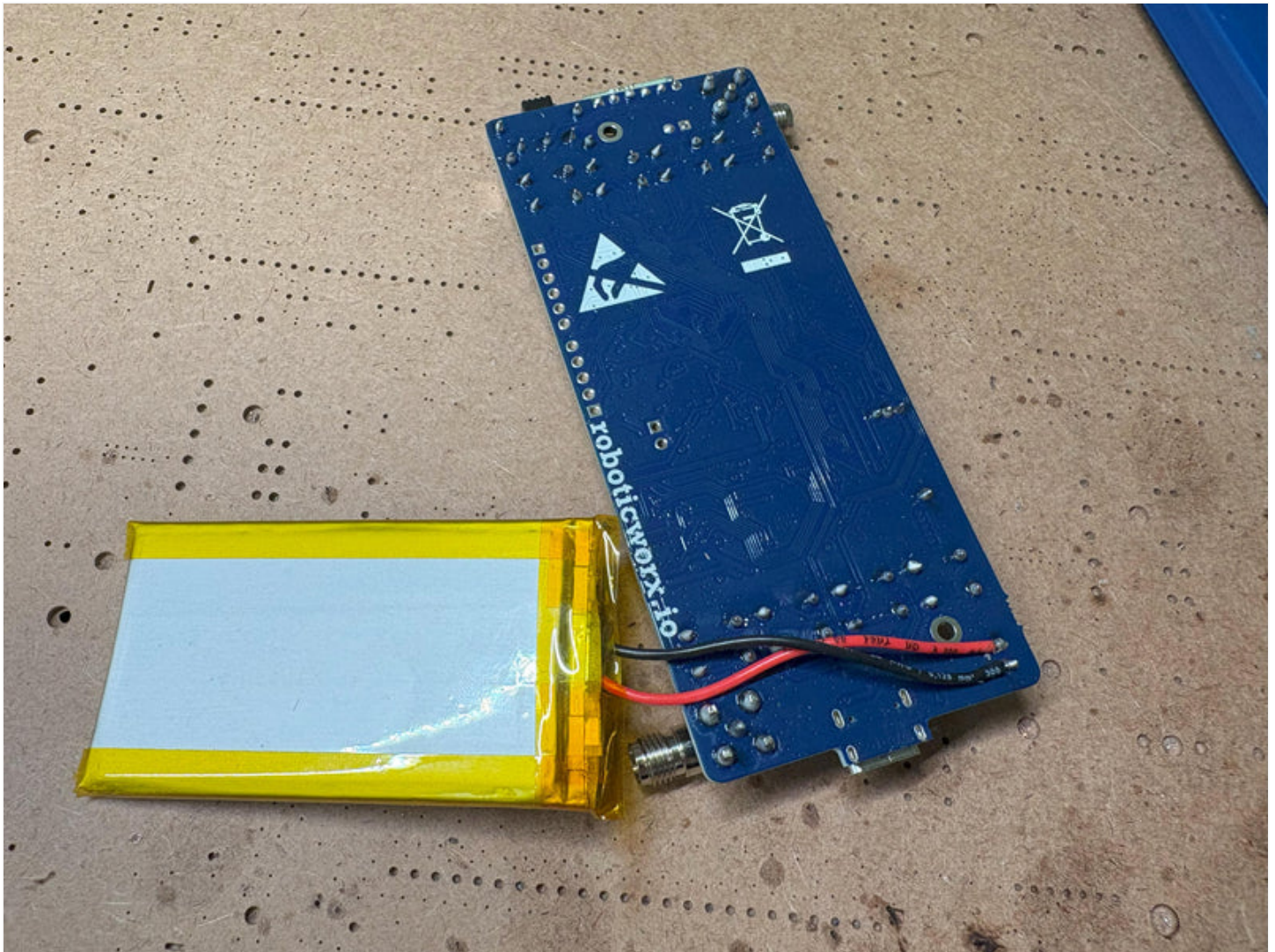
The logo-ed STL 3D printer files can be found [here for free on my Thingiverse!](#)

If you're interested in obtaining the [logo-less STL files and the editable STEP files](#), please consider [becoming a plus subscriber](#)! I don't use ads, so I rely on your support! You will also get [these additional benefits](#).

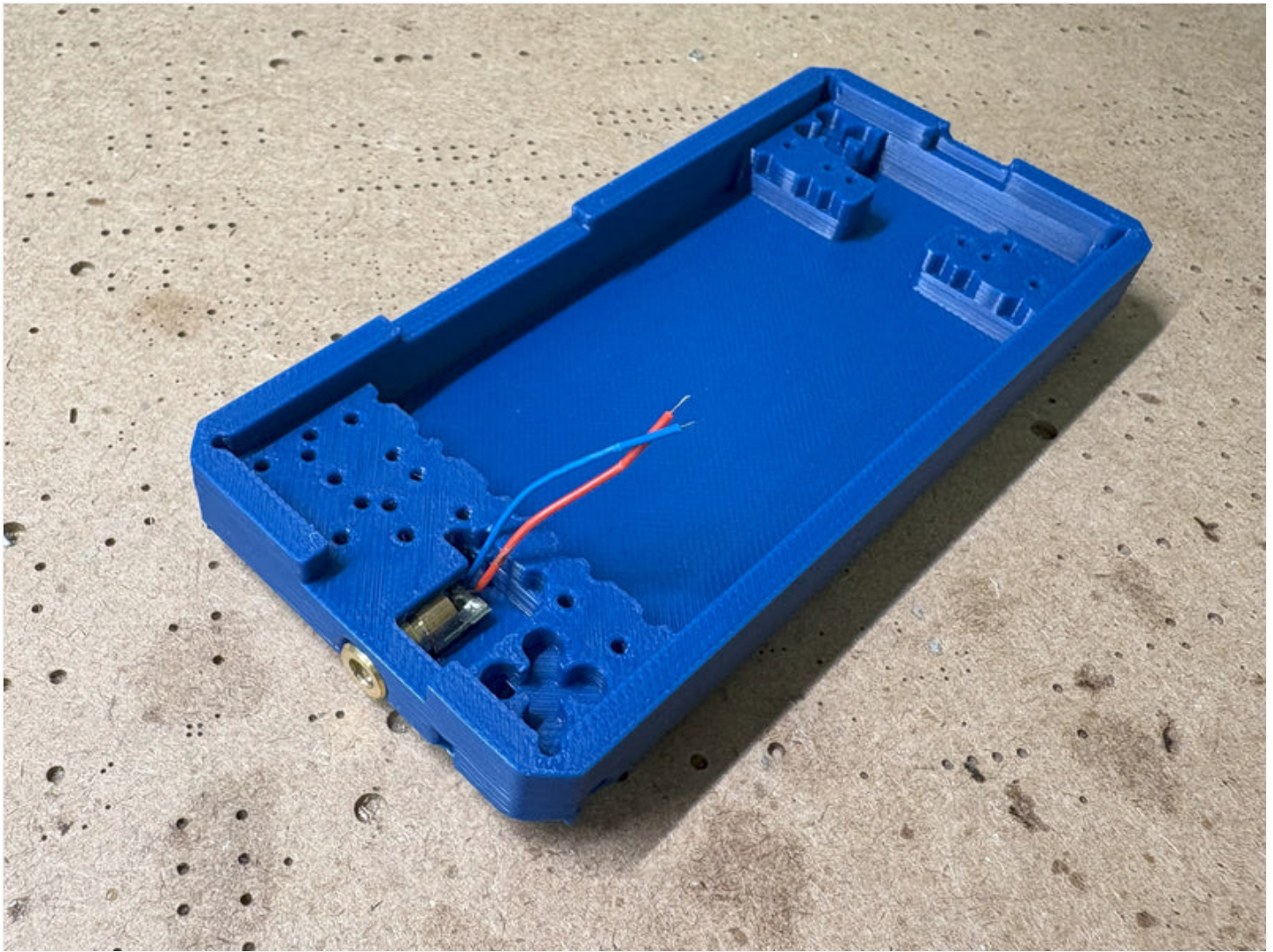
In addition, you can also use DualCast to interface with my light switcher gizmo and door locker from my [Gateway project](#). The CAD for these is available [under "Gateway Smartwatch"](#) for [plus subscribers](#).



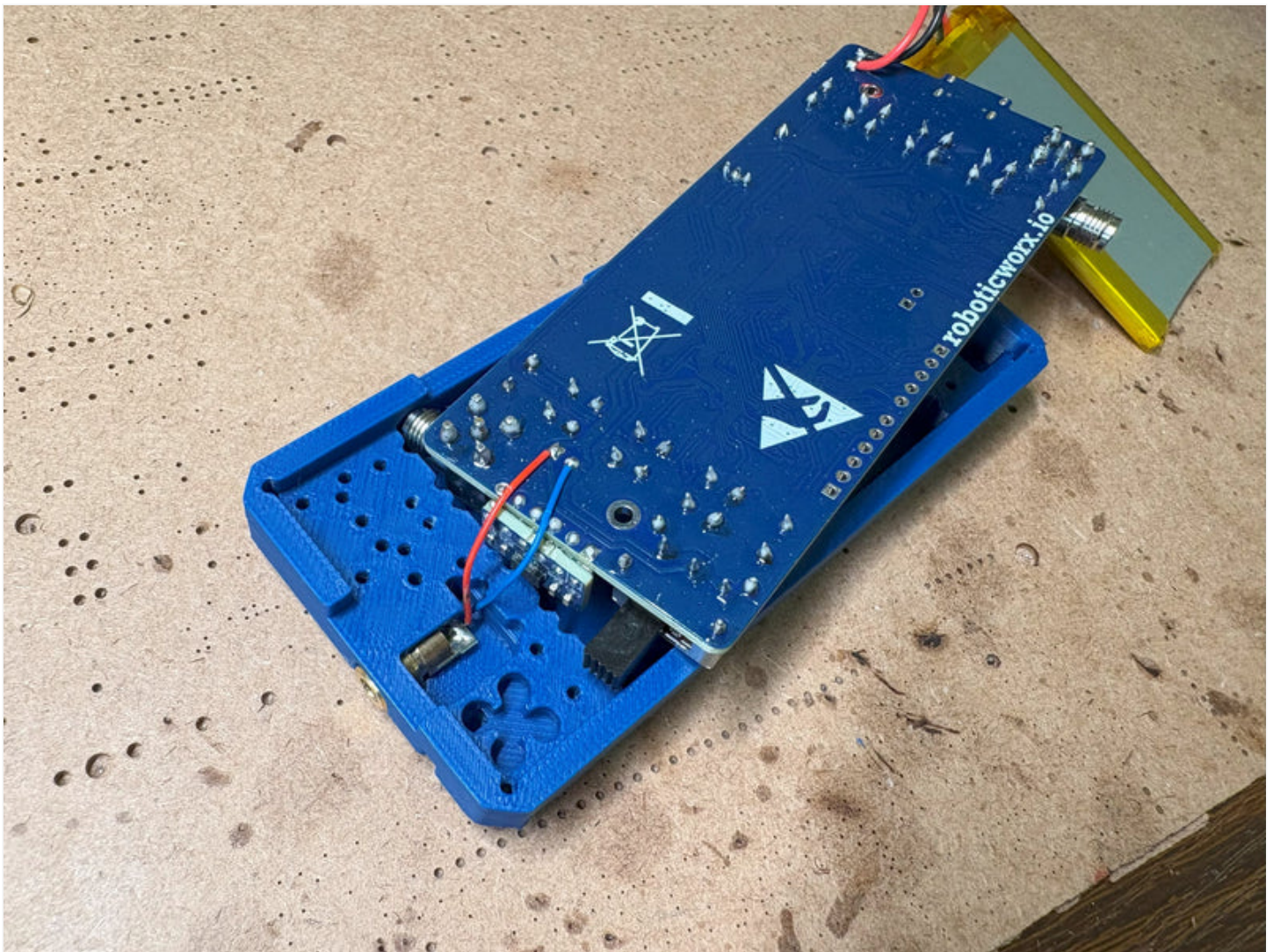
First, grab your assembled PCB and solder on the battery.

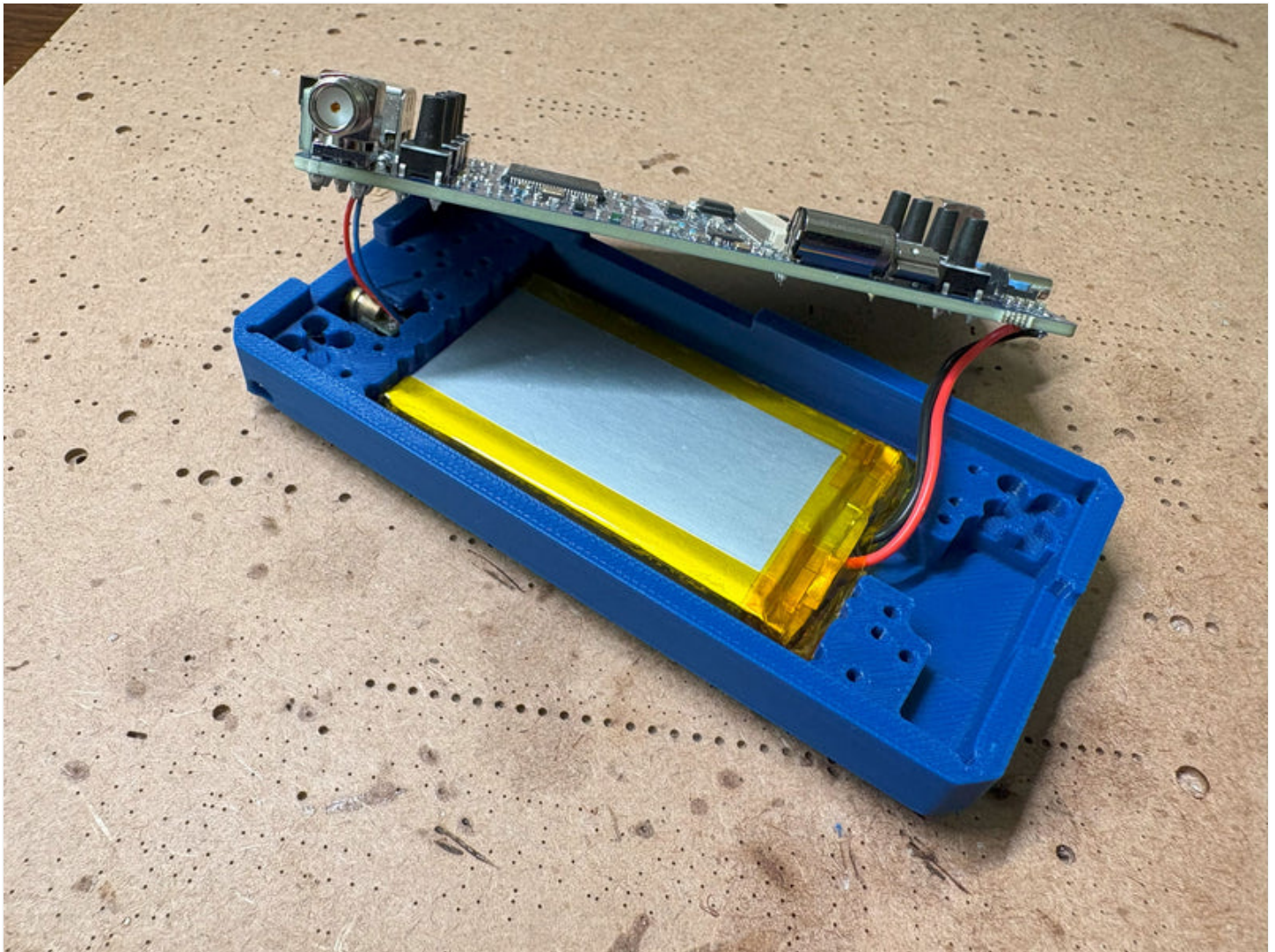


Then, push through a [laser diode](#) and trim the ends about two-thirds. You may also want to go ahead and focus the laser by twisting the lens cap with power applied, since sometimes the dot isn't very well focused.



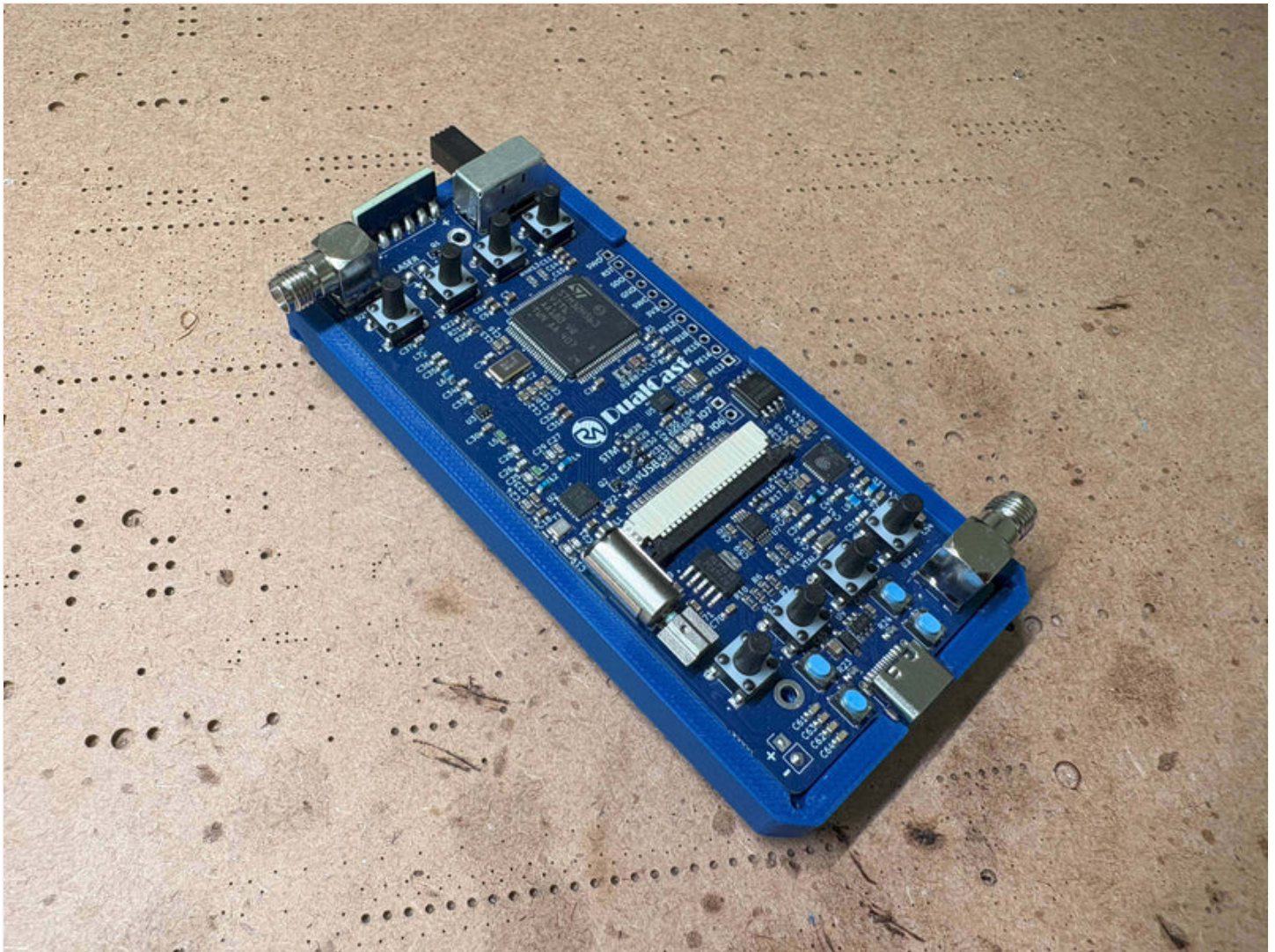
After that, solder it to the board.





Next, push the PCB in while doing your best to position the wires into the spaces on the 3D-print. It should be as flush as possible.

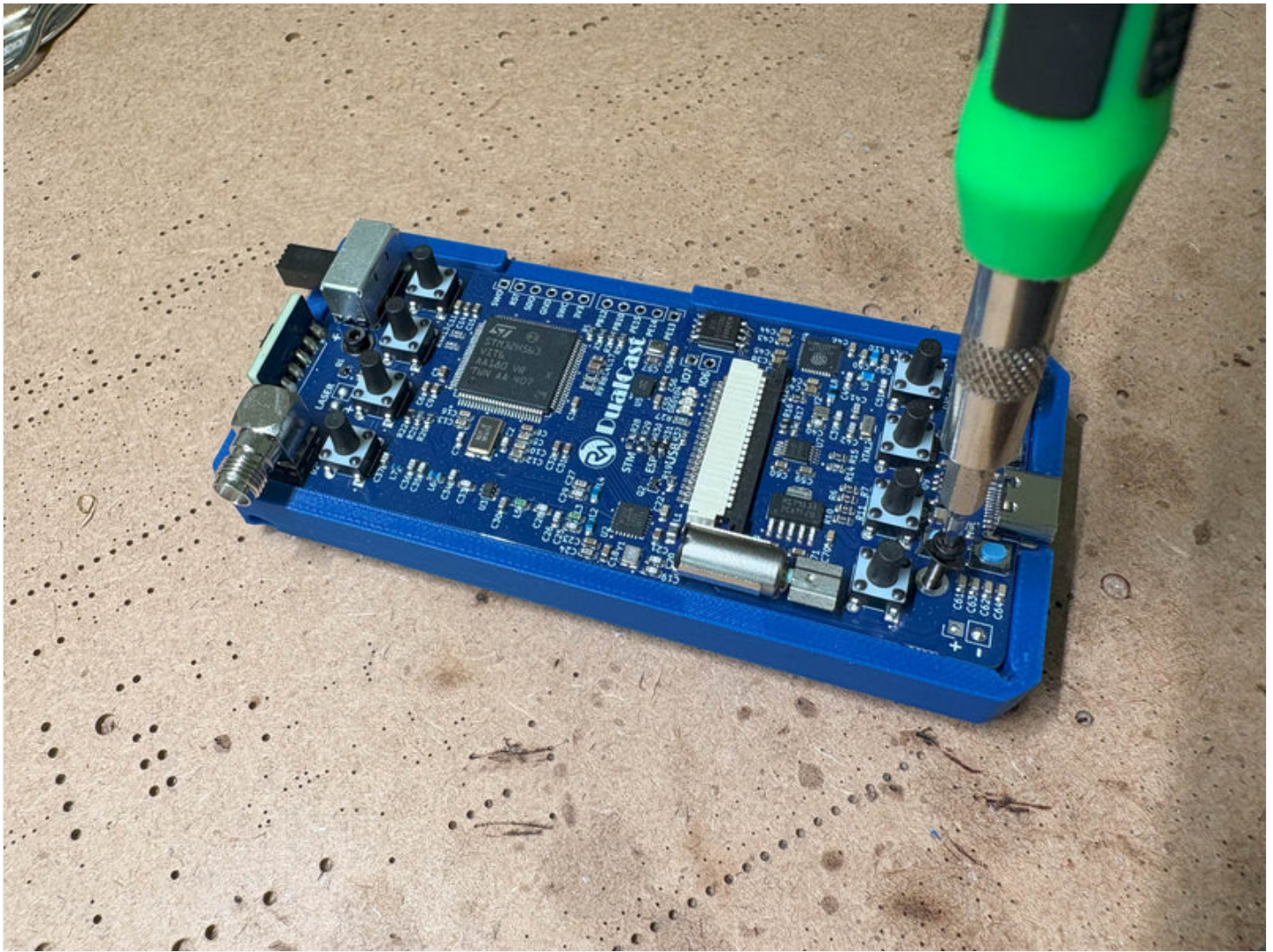
AGAIN: Do not power it on and then press buttons without the antennas attached or the ESP32/SX1262 could be damaged!



Then, flip it over and push in two [M2 nuts](#).



Using two [M2x12 bolts and some washers](#), you can then mount the PCB to the casing.



Next, grab some more M2 nuts and push them into the corners.



When that's done, you can gently flip it over and push in the LCD. Make sure to pop out the connector piece before pushing it in, and then pop it back to secure it in place.



Then, grab the top part of the casing and push the LCD through the opening. From there, apply some glue, push it down, and let it sit for a few hours.

After the glue has dried, push the top piece down and then use some tweezers or something else thin to push the FFC (Flat Flexible Cable) of the LCD toward the top. Otherwise, it will block the haptic motor from spinning. You can also go ahead and pop in the buttons through the top. Feel free to use a dab of glue if you're concerned they might fall out.

Next, screw in four [M2x20 bolts](#) in the corners, feeding into the nuts you placed on the back. Make sure to not let up on the top piece, since you want to make sure the FFC stays bent toward the top.

Great! Then, just screw in the antennas and you're done!

Using the Receiver

Here I'll go into some details about setting up the receiver for versatile use in whatever you desire to use this project for!



Note: The 6 pins above the GPIOs are for connecting a LiDAR should you ever want to add presence detection.

As I mentioned earlier, all this receiver does is receive the signal (a number) sent from DualCast, then sets the corresponding GPIOs high to match it in binary. Since there are 5

GPIOs, this will range from 0-31 ($2^5 - 1$), meaning there are a possible 31 different commands that can be received (not counting 0). *If you're not familiar with binary, a list of the binary equivalents of decimal numbers [can be found here](#).*

Binary Numbers List

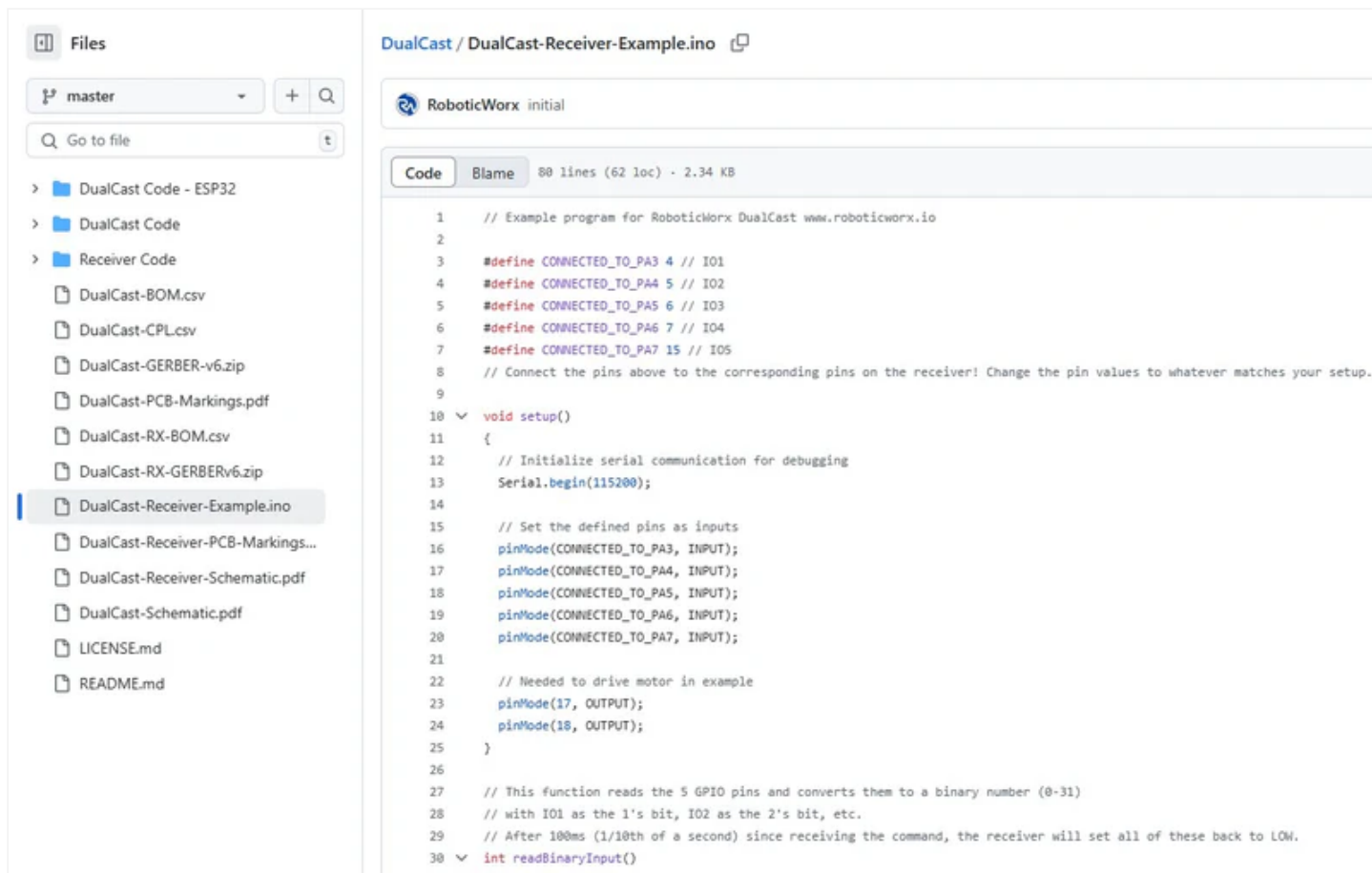
Binary numbers are written in the form of zero's and one's only. A list of binary numbers up to 10000 is given below. In the given table, we have shown decimal numbers with the corresponding binary numbers.

List of Binary Numbers from 1 to 100

A list of binary numbers from 1 to 100 are given below:

No.	Binary Number
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
...	...

So, all you need to do to use the receiver is connect the 5 GPIOs on the receiver to whatever you're familiar with using (Arduino UNO, ESP32, etc.) then monitor these pins for the corresponding command. When the command you're looking for is received, you can have it do whatever in Arduino IDE which is probably a bit more comfortable than using C in CubeIDE (*although you're more than welcome to modify the receiver [source code](#) directly if you want to.*). To help streamline this process, I also created a basic [Arduino receiver sketch to reference here](#).



The screenshot displays the RoboticWorx IDE interface. On the left, the 'Files' panel shows a project structure with a 'DualCast' folder containing a 'Receiver Code' subfolder. The file 'DualCast-Receiver-Example.ino' is selected. The main editor area shows the code for this file, which is an Arduino sketch. The code includes comments and defines pins for 5 inputs (PA3-PA7) and 2 outputs (17, 18). It initializes serial communication at 115200 baud, sets pin modes, and includes a setup function and a readBinaryInput function.

```
1 // Example program for RoboticWorx DualCast www.roboticworx.io
2
3 #define CONNECTED_TO_PA3 4 // IO1
4 #define CONNECTED_TO_PA4 5 // IO2
5 #define CONNECTED_TO_PA5 6 // IO3
6 #define CONNECTED_TO_PA6 7 // IO4
7 #define CONNECTED_TO_PA7 15 // IO5
8 // Connect the pins above to the corresponding pins on the receiver! Change the pin values to whatever matches your setup.
9
10 void setup()
11 {
12     // Initialize serial communication for debugging
13     Serial.begin(115200);
14
15     // Set the defined pins as inputs
16     pinMode(CONNECTED_TO_PA3, INPUT);
17     pinMode(CONNECTED_TO_PA4, INPUT);
18     pinMode(CONNECTED_TO_PA5, INPUT);
19     pinMode(CONNECTED_TO_PA6, INPUT);
20     pinMode(CONNECTED_TO_PA7, INPUT);
21
22     // Needed to drive motor in example
23     pinMode(17, OUTPUT);
24     pinMode(18, OUTPUT);
25 }
26
27 // This function reads the 5 GPIO pins and converts them to a binary number (0-31)
28 // with IO1 as the 1's bit, IO2 as the 2's bit, etc.
29 // After 100ms (1/10th of a second) since receiving the command, the receiver will set all of these back to LOW.
30 int readBinaryInput()
```

With this, you'll be able to use the LoRa receiver for any custom projects you want and communicate with it easily on your DualCast!

Using DualCast

Here I'll explain what all the different buttons do and how to use DualCast like a pro! If you just got here, the info on using the receiver is above ^. I'll go ahead and label the buttons as below to make everything easier to explain.



Numbers will flip with orientation!

Okay, so first things first. Turning DualCast on while upright (like above) will put it into LoRa mode. From here, you have the following button options:

- Button 1 will show your encryption/decryption key and iterate the selected hexadecimal character.
- Button 2 will show your encryption/decryption key and change which hexadecimal character is currently selected.
 - Use these two buttons to update your key, selecting and then changing it one character at a time.
- Button 3 will subtract 4 from all number commands at the bottom.
- Button 4 will add 4 to all number commands at the bottom.
 - These two are used to change which number (as a command) you want to send to the receiver. For example, if the receiver is listening for a 2, or 8, or 12, etc., you can update the range here.

- Buttons 5-8 will just send whichever number is above it. For example, if there is a 1 above button 5, when you press button 5 it will send a 1.

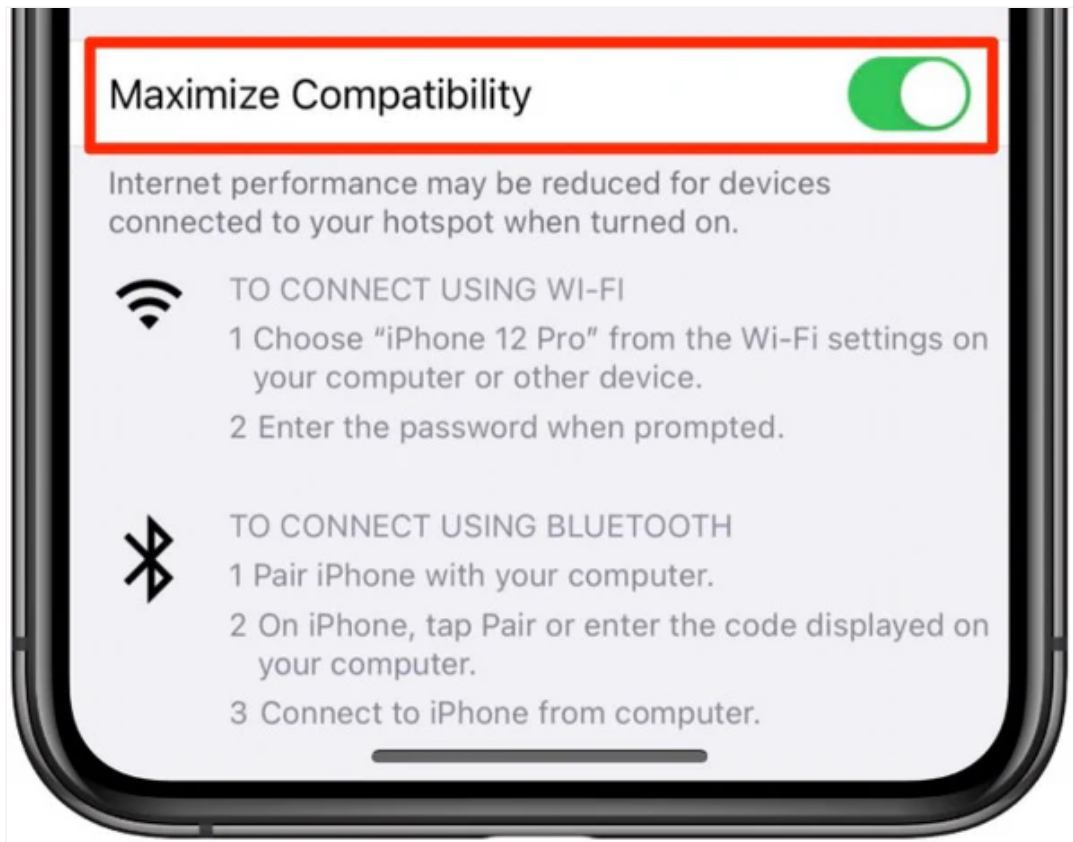


On to Wi-Fi mode! To put it into this mode, simply turn DualCast off, flip it 180 degrees, and then turn it back on.



New button numbers.

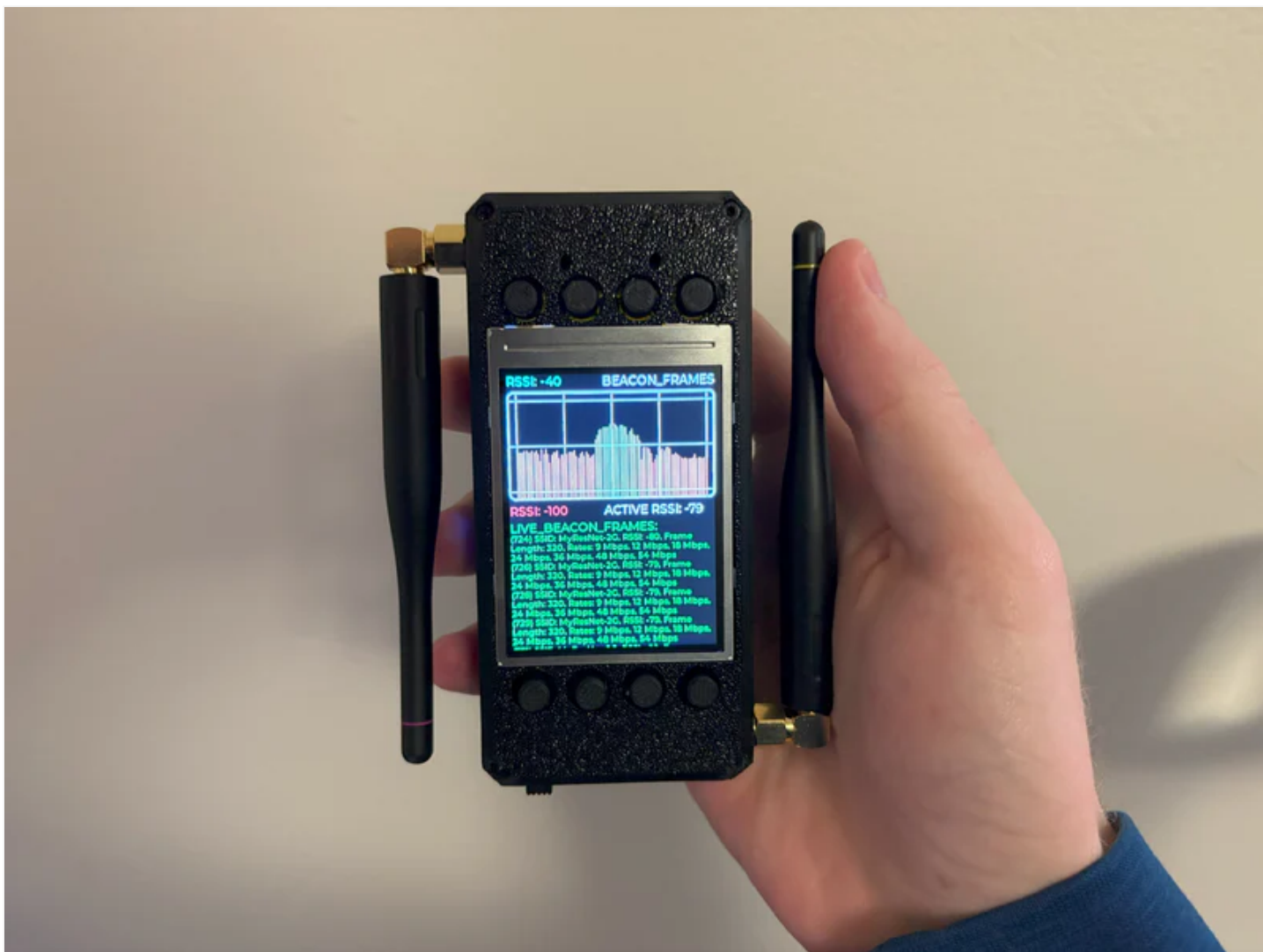
To start a Wi-Fi scan, simply select **any TOP button (1-4)**. If you select a bottom one, the program will crash and you'll need to power cycle to reset it. I should also note here that if you start the scan and nothing shows up after a few seconds, then **it means there are no 2.4GHz networks around to analyze**. Try moving somewhere else and trying again. As a good test, if you have an iPhone you can turn on a hotspot with "Maximize Compatibility" selected (makes it 2.4GHz compatible) and it will be visible on DualCast. From there, you can connect to the hotspot with your PC, etc., and then see all the data frames your computer or other device is sending which is pretty neat. (Generate some traffic!)



After starting a scan:

- Button 5 will scroll up one network.
- Button 6 will scroll down one network.
- Button 7 will select the network you've currently scrolled to (the network at the top).
 - As a helpful identifier, the number at the top right of the screen is the network currently set to be selected.

After selecting the given network with button 7, DualCast will be filtered to only listen for packets to/from that network. The only still active button is now button 8 which you can use to toggle between data and beacon frames which I discussed earlier.



Awesome! The only thing now is the LiDAR. It's pretty easy to select, all you have to do is hold button 4 and then press button 3 while it's held (and in LoRa mode/upright orientation) to turn on the VL53L4X LiDAR. In this mode, you get the current distance measured as well as some additional data to ensure measurement accuracy such as the ambient light, tolerance, min and max.



To exit LiDAR mode, just cycle power. If you ever forget what these buttons do, there is also a built-in QR code on DualCast which can be easily accessed by holding button 4 and pressing button 2 while held (in LoRa mode) similar to activating the LiDAR (but one button to the left).



This was a tough picture to get since my phone kept trying to read the QR haha.

BOM

This is the Bill of Materials for my DualCast project!

I'll put everything that you need to have here so that you don't have to go scrolling around looking for the links I sprinkled throughout the article.

- **All the CAD for this project can be found [here on my Thingiverse](#).**
- **Everything else including the Gerber files, part list, etc. can be found [here on my GitHub](#).**
- **The editable STEP files, logo-less STL files, PCB STEP file, and editable KiCad PCB files are [available here](#) to [plus subscribers](#). Thanks so much for your support!**
- [Laser diodes](#)

- [ESD-proof IC storage container](#)
- [Optional receiver battery](#)
- [STLink](#) (many options)
- If you don't already have them:
 - [Mini soldering iron](#) (what I use, but any will do)
 - [Solder paste](#)
 - [Solder](#) (my roll of choice)
 - [Hot plate](#)
 - [Hot air rework gun](#) (great for fixing broken or misaligned ICs!)
 - Some [nuts & bolts](#)
 - [Basic screwdriver kit](#) (one of many options)

Disclosure: These are affiliate links. I get a portion of product sales at no extra cost to you.



Thanks so much for reading! I hope this was a helpful and informative article. If you decide to do the build, please feel free to leave any questions in the comments below. If not, I hope you were still able to enjoy reading and learn something new!

Have constructive criticism or a suggestion for a future project? I'm always looking to improve my work. Leave it in the comments! Until next time.

Be sure to follow me on [Instagram](#)! If you want to learn more cool stuff, I also highly recommend [Branch Education](#) and [Kurzgesagt](#) on YouTube. :)

If you feel this read was worth at least \$5, please consider [becoming a plus subscriber](#). I don't use ads, so I rely on your support to keep going. *This is a quick read for you, but I've been working hard on this for months!* You will also receive [the following benefits](#).